Ligero と zk-STARK を

基礎から学ぶ

ゼロ知識証明の入門から、Ligero・zk-STARK の構造、

小例・比較・ユースケースまでを一冊で整理した教科書的解説

ゼロ知識証明の代表的な二系統である Ligero と zk-STARK を、直観・数学・具体例・ユースケースの順に、初学者から学部生までを想定して教科書的に解説する Zenn Book。

収録内容
零知識証明の基本 / 前提数学 / Ligero の全体像と内部構造 / Ligero の手計算例 / zk-STARK の全体像 / AIR / ALI / FRI / zk-STARK の手計算例 / 比較 / ユースケース / 学習ロードマップ / 用語集

PDF 版

2026-04-06

目次

はじめに：Ligero と zk-STARK をどう学ぶか **1**
本書の狙い 1
読者像 1
まず押さえたい全体像 2
「教科書的」に読むための姿勢 2
本書の道順 2
本書で使う約束 2
最初に結論だけ言うと 4
この章のまとめ 4

第 1 章 零知識証明の土台 **5**
1.1 statement と witness 5
1.2 三つの基本性質 6
1.3 proof と argument の違い 6
1.4 対話型と非対話型 7
1.5 公開コインと透明性 7
1.6 PCP, IOP, IOPP を最小限だけ 8
1.7 なぜ「全部見ない」のに信用できるのか 8
1.8 典型的な誤解 8
1.9 小さな例：パスワードを見せずに知っていることを示す 9
1.10 この章のまとめ 9
練習問題 9

第 2 章 数学と道具の準備 **10**
2.1 有限体とは何か 10
2.2 多項式は “構造を持った関数” である 11
2.3 補間：値の表から多項式を戻す 11
2.4 Reed–Solomon 符号の直観 11
2.5 “近い” とは何か：距離の考え方 12
2.6 低次数性がなぜ嬉しいのか 13
2.7 ランダム点検査の基本原理 13
2.8 Merkle 木：大きな表へのコミットメント 13
2.9 オラクルモデルとは何か 14
2.10 低次数拡張（Low-Degree Extension; LDE） 14
2.11 Interleaving の直観 15
2.12 ここまでの道具がどう使われるか 15
2.13 この章のまとめ 15
練習問題 16

第 3 章 Ligero の全体像 **17**
3.1 Ligero とは何か 17
3.2 どうして平方根が出てくるのか 17
3.3 Ligero の設計思想 18
3.4 Ligero の全体フロー 18
3.5 何が嬉しいのか 19

3.6 原論文の代表的な結果をどう読むか . . . 19
3.7 Ligero はどんな問題に向くか . . . 20
3.8 Ligero が万能ではない理由 . . . 20
3.9 Ligero を一文で言うなら . . . 20
3.10 この章のまとめ . . . 21
練習問題 . . . 21

第 4 章 Ligero の中身を分解する 22
4.1 出発点：回路満足性 . . . 22
4.2 witness と配線値を行列に並べる . . . 22
4.3 各行を RS 符号で符号化する . . . 23
4.4 行の “符号語らしさ” をどう検査するか . . . 24
4.5 なぜランダム線形結合が効くのか . . . 24
4.6 線形制約の検査 . . . 25
4.7 乗算制約の検査 . . . 25
4.8 回路全体はどう処理されるか . . . 26
4.9 零知識はどう入るのか . . . 26
4.10 コミットメントと非対話化 . . . 27
4.11 Ligero の内部を一枚絵にすると . . . 27
4.12 ありがちな疑問 . . . 27
4.13 この章のまとめ . . . 27
練習問題 . . . 29

第 5 章 Ligero を小さな回路で追う 30
5.1 証明したい主張 . . . 30
5.2 回路に直す . . . 30
5.3 行列に並べる . . . 31
5.4 各行を多項式として見る . . . 31
5.5 RS 符号語にする . . . 31
5.6 行の membership test をやってみる . . . 32
5.7 もし不正な行が混じっていたら？ . . . 33
5.8 線形制約をこの例で見る . . . 34
5.9 乗算制約をこの例で見る . . . 34
5.10 この小例で見えたもの . . . 35
5.11 では平方根オーダはどこから来るか . . . 35
5.12 ブール回路との関係 . . . 36
5.13 この章のまとめ . . . 36
練習問題 . . . 36

第 6 章 zk-STARK の全体像 37
6.1 STARK という名前 . . . 37
6.2 STARK が狙う問題 . . . 37
6.3 STARK の基本直観 . . . 38
6.4 STARK の全体フロー . . . 38
6.5 透明性とは何か . . . 38
6.6 “post-quantum secure” と言われる理由 . . . 39
6.7 STARK の代表的な長所 . . . 39
6.8 STARK の弱み・難しさ . . . 40
6.9 原論文の象徴的な例：DNA プロファイル照合 . . . 40
6.10 Ligero との第一印象の違い . . . 40
6.11 この章のまとめ . . . 41
練習問題 . . . 41

第 7 章 AIR：計算を状態遷移として書く **42**
7.1 なぜ AIR が必要なのか 42
7.2 実行トレースという見方 42
7.3 フィボナッチ列で AIR を作る 43
7.4 遷移規則を多項式で書く 43
7.5 境界条件（boundary constraints） 44
7.6 AIR を “局所制約” と見る 44
7.7 列を多項式として補間する 45
7.8 AIR 制約をトレース多項式に持ち込む 45
7.9 低次数が重要な理由を AIR の文脈で見る 45
7.10 AIR の一般形 46
7.11 AIR は回路と何が違うのか 46
7.12 フィボナッチ例を AIR としてまとめる 47
7.13 この章のまとめ 47
練習問題 47

第 8 章 FRI と ALI：低次数性をどう証明するか **48**
8.1 まず役割分担を先に言う 48
8.2 LDE：なぜ大きい領域で評価し直すのか 48
8.3 Merkle 木：大きなオラクルを固定する 49
8.4 ALI：多数の制約をどうやってまとめるか 49
8.5 composition polynomial という見方 50
8.6 低次数性を本当にどう調べるか：FRI 50
8.7 FRI の直観：大きな問題を半分にたたむ 50
8.8 FRI をもう少しだけ具体的に 51
8.9 FRI は何を証明し、何を証明しないか 51
8.10 零知識は STARK でどう入るか 52
8.11 Fiat–Shamir と非対話化 52
8.12 STARK の各部品を Ligero と見比べる 52
8.13 この章のまとめ 53
練習問題 53

第 9 章 zk-STARK をフィボナッチで追う **54**
9.1 証明したい主張 54
9.2 実行トレースを作る 54
9.3 AIR を書く 54
9.4 列を多項式にする 55
9.5 小領域から大領域へ：LDE 55
9.6 Merkle でコミットする 55
9.7 制約を一つにまとめる 56
9.8 もし最終値を偽装したらどうなるか 56
9.9 FRI をこの例で見る 57
9.10 検証者は具体的に何を見るのか 57
9.11 なぜ少数点で十分なのか 57
9.12 零知識はこの例でどう効くか 58
9.13 フィボナッチから実運用への橋渡し 58
9.14 この章のまとめ 58
練習問題 58

第 10 章 Ligero と zk-STARK を比較する **59**
10.1 まず一言で違いを言う 59
10.2 比較表 59

10.3 何を圧縮しているのか . . . 59
10.4 回路 vs トレース . . . 60
10.5 ランダム線形結合という共通骨格 . . . 60
10.6 低次数性の使い方の違い . . . 61
10.7 証明サイズ・検証時間・証明時間の感覚 . . . 61
10.8 透明性の意味では両者は近い . . . 62
10.9 零知識の入れ方の違い . . . 62
10.10 実装のしやすさという意味での違い . . . 62
10.11 どんなときに Ligero を考えるか . . . 63
10.12 どんなときに STARK を考えるか . . . 63
10.13 “どちらを選ぶか” の雑な指針 . . . 63
10.14 この章のまとめ . . . 63
練習問題 . . . 64

第 11 章 ユースケース：どこで役立つのか 65
11.1 ユースケースを三種類に分ける . . . 65
11.2 パスワード・ハッシュ・認証 . . . 65
11.3 秘匿監査・外部委託計算 . . . 66
11.4 大規模状態更新とブロックチェーン . . . 66
11.5 StarkEx と Starknet は何を示しているか . . . 67
11.6 DNA データベース例が示すもの . . . 67
11.7 Ligero の使いどころを丁寧に捉える . . . 67
11.8 多数インスタンスの一括証明 . . . 68
11.9 ゼロ知識証明は “秘密保持” だけの技術ではない . . . 68
11.10 どんなときに使わない方がよいか . . . 68
11.11 実務的な選び方 . . . 69
11.12 この章のまとめ . . . 69
練習問題 . . . 69
参考メモ . . . 69

第 12 章 学習ロードマップと参考文献 70
12.1 どこまで理解できていれば十分か . . . 70
12.2 おすすめの読み順 . . . 70
12.3 原論文を読むときのコツ . . . 71
12.4 参考文献（最重要） . . . 71
12.5 公式資料・公式ドキュメント . . . 72
12.6 自習課題のすすめ . . . 72
12.7 よくある学習上のつまずき . . . 73
12.8 この章のまとめ . . . 73
12.9 参考リンク集（検索しやすい題名付き） . . . 73

付録 用語集 75
AIR（Algebraic Intermediate Representation） . . . 75
ALI（Algebraic Linking IOP） . . . 75
argument . . . 75
codeword（符号語） . . . 75
completeness（完全性） . . . 75
commitment（コミットメント） . . . 75
composition polynomial . . . 75
execution trace（実行トレース） . . . 75
Fiat–Shamir 変換 . . . 76
FRI（Fast Reed–Solomon Interactive Oracle Proof of Proximity） . . . 76

Hamming distance（ハミング距離） 76
interleaved Reed–Solomon code 76
interactive proof / interactive argument 76
IOP（Interactive Oracle Proof） 76
IOPP（Interactive Oracle Proof of Proximity） 76
LDE（Low-Degree Extension） 76
Merkle tree（Merkle 木） 76
oracle 77
PCP（Probabilistically Checkable Proof） 77
proof 77
public-coin 77
random oracle model 77
Reed–Solomon code 77
scalability（スケーラビリティ） 77
soundness（健全性） 77
statement 77
transparent 77
witness 78
zero-knowledge（零知識） 78

はじめに：Ligero と zk-STARK をどう学ぶか

本書は、ゼロ知識証明の代表的な二つの流れ ——**Ligero** と **zk-STARK**—— を、**初学者から学部生でも追えること**を目標に、できるだけ教科書的に整理して解説するものです。

両者はどちらも「**秘密を見せずに、計算が正しいことだけを示す**」ための仕組みですが、内部の考え方はかなり異なります。

- **Ligero** は、回路の満足性を**行列と符号**に落として、ランダムな線形結合と少数の点の検査で不正を見つける系統です。
- **zk-STARK** は、計算全体を**実行トレース**として眺め、それが**低次数多項式**として一貫していることを示す系統です。

この違いは、単なる実装差ではありません。
「何を証明対象として圧縮するか」という思想そのものが違います。

本書の狙い

本書の狙いは、次の三つです。

1. **直観を作る**
   いきなり定義や定理に入るのではなく、「なぜそんな検査で十分なのか」を先に掴みます。

2. **最低限の数学に橋を架ける**
   有限体、多項式、Reed–Solomon 符号、Merkle 木、低次数検査など、必要な道具を一つずつ導入します。

3. **プロトコルを "追える" ようにする**
   Ligero では小さな回路、zk-STARK ではフィボナッチ列の状態遷移を題材に、証明の流れを順に追います。

読者像

想定している読者は、次のような人です。

- ゼロ知識証明を名前しか知らない初学者
- 暗号理論・情報理論・計算理論を学び始めた学部生
- 「SNARK や STARK をニュースで聞くが、中身の骨格はまだ曖昧」という人
- 研究の入り口として、原論文を読む前に全体像を掴みたい人

逆に、本書は次のことを**主目的にはしません**。

- 実装チューニングの細部
- 既存ライブラリの API の解説
- Cairo や専用 VM の開発チュートリアル
- 最新の L2 エコシステムの網羅的紹介

そうした話題には最終章の学習ロードマップで触れます。

まず押さえたい全体像

Ligero と zk-STARK を、極端に単純化して比べるとこうなります。

観点	Ligero	zk-STARK
証明したいもの	回路の制約が満たされていること	長い実行トレースが正しい遷移に従うこと
主役	行列・符号・ランダム線形結合	実行トレース・AIR・FRI
典型的な直観	「たくさんの制約を一つにまとめて spot-check する」	「巨大な計算を低次数多項式として圧縮する」
長所	単純な部品で構成しやすい、サブリニア通信	透明性・大規模計算へのスケーラビリティ
苦手な点	超巨大計算では STARK 系に比べて見劣りしやすい	証明サイズや実装が重くなりやすい

本書を読み終えるころには、上の表を**自分の言葉で説明できる**ようになるはずです。

「教科書的」に読むための姿勢

ゼロ知識証明は、最初に触れると次の二点でつまずきやすいです。

1. “全部は見ていないのに、なぜ信用できるのか”

これは確かに直観に反します。
しかし、ランダム性と代数的構造を組み合わせると、「嘘をつくなら多くの場所で矛盾が出る」ように設計できます。すると、少数のランダム検査でも不正を高確率で見つけられます。

2. “多項式がなぜそんなに重要なのか”

多項式は、単なる関数ではなく、**少数の評価値から強い構造が回復できる対象**です。
「低次数である」という条件は、情報を圧縮しつつ、不正を検出しやすくする鍵になります。

Ligero でも zk-STARK でも、この二点が本質です。
違いは、その構造をどこに埋め込むかです。

本書の道順

おすすめの読み方は次の通りです。

- まったく初めてなら、**第 1 章・第 2 章から順番に**読んでください。
- 数学に慣れているなら、**第 3 章（Ligero）または第 6 章（zk-STARK）から**先に読んでも構いません。
- 研究の見取り図が欲しいなら、まず**第 10 章の比較**を先に読んでから戻るのも有効です。

本書で使う約束

- 公開された主張を **statement**、それを成り立たせる秘密情報を **witness** と呼びます。
- 「証明」という言葉は日常語として使いますが、厳密には **proof** と **argument** を区別します。
  この違いは第 1 章で説明します。
- 数式は必要最小限に絞りますが、避けるべきでないところではきちんと書きます。
- 直観的説明と厳密な意味が少しズレる箇所では、そのことを明示します。

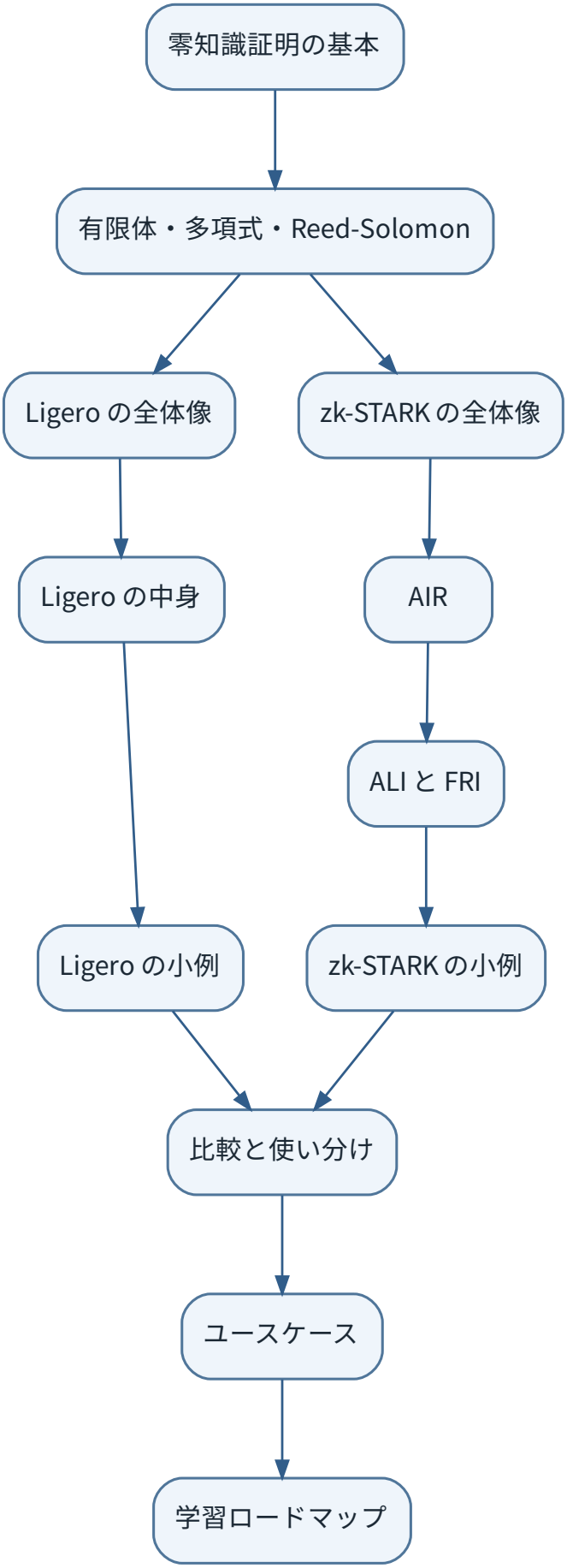

図 1: 本書の学習ロードマップ

最初に結論だけ言うと

Ligero と zk-STARK は、どちらも

- trusted setup を避けやすく、
- ハッシュ関数や符号理論を中核に置き、
- 巨大な計算を少ない検査に圧縮する

という点で共通しています。

しかし、

- **Ligero** は「回路制約をうまく束ねて検査する」思想が強く、
- **zk-STARK** は「長い実行を多項式として丸ごと検査する」思想が強い、

という違いがあります。

この違いが、証明サイズ、検証時間、実装のしやすさ、ユースケースを分けます。

この章のまとめ

- Ligero と zk-STARK は、どちらもゼロ知識証明の重要な流れである。
- ただし内部構造はかなり違う。
- Ligero は**行列・符号・ランダム線形結合**の世界、zk-STARK は**実行トレース・AIR・FRI** の世界で理解すると見通しがよい。
- 次章では、その前提となる零知識証明の基本概念を整理する。

第 1 章 零知識証明の土台

ゼロ知識証明は、一言で言えば

> 「**秘密そのものを出さずに、秘密を知っていること、あるいは秘密が条件を満たすことだけを示す**」

ための仕組みです。

ただし、この一言では大事な点が隠れています。
「何を誰に示すのか」「どの程度まで騙せないのか」「なぜ秘密が漏れないのか」を、順に整理しましょう。

1.1 statement と witness

ゼロ知識証明では、普通は次の形の問題を考えます。

- 公開される主張：`statement` x
- 秘密の補助情報：`witness` w

そして「関係」$R(x, w)$ が定義されていて、

$$R(x, w) = 1$$

なら、「w は x の正しい witness である」と言います。

例 1：ハッシュ前像

公開された値 h に対し、

- statement: $x = h$
- witness: $w = m$
- relation: $R(h, m) = 1 \iff H(m) = h$

です。
ここで示したいことは「**私は m を知っている**」であって、「m 自体を公開する」ではありません。

例 2：数独

完成済みの数独の盤面を witness とし、

- 与えられた数字は変えていない
- 各行・各列・各 3×3 ブロックに 1〜9 が一回ずつ現れる

という relation を満たすことを示すこともできます。
このとき、解答そのものは秘密にしたいかもしれません。

例 3：ブロックチェーンの状態更新

L2 システムでは、

- statement: 「この新しい状態ルートは正しい」
- witness: 実際の取引列、各アカウントの更新過程、中間状態

のように考えます。
ユーザやバリデータは中間計算を全部やり直さずに、最終更新が正しいと納得したいわけです。

1.2 三つの基本性質

ゼロ知識証明を理解するうえで最も重要なのは、**完全性・健全性・零知識性**の三つです。

完全性（completeness）

主張が本当に正しく、正直な証明者が正しい witness を持っているなら、検証者は受理する。

直観的には、

> **本当のことを言っている人が不当に落とされない**

という性質です。

健全性（soundness）

主張が偽であるなら、不正な証明者がどれほど工夫しても、検証者を騙して受理させる確率は小さい。

つまり、

> **嘘つきが通ってしまう確率を十分小さく抑える**

という性質です。

ゼロでなく「小さい」と言うのは、現代の多くの証明系が確率的だからです。
たとえば健全性誤差が 2^{-80} なら、偽の主張が受理される確率は高々 2^{-80} とみなします。

零知識性（zero-knowledge）

検証者は、主張が真であること以外の有益な情報を得ない。

ここでの厳密な定式化は、「**シミュレータ**」という仮想的なアルゴリズムが存在し、真のやりとりを見たのと区別できない transcript を生成できる、というものです。

直観的には、

> **見た目には会話があったが、実は witness を知らなくても、その会話らしいものを作れてしまう**

なら、会話そのものから新しい秘密情報は得られていない、という考えです。

1.3 proof と argument の違い

日常語ではどちらも「証明」と言いたくなりますが、暗号では区別が重要です。

Proof

どれほど計算能力の大きい相手に対しても健全性が成り立つもの。
つまり、**情報理論的**あるいは**無条件**に近い強さを持つことがあります。

Argument

健全性が、ある計算問題の難しさに依存するもの。
たとえばハッシュ関数の衝突耐性や離散対数問題の難しさに依存します。

現代の実用的なゼロ知識システムの多くは、厳密には **argument** です。
Ligero も zk-STARK も、実用形では通常 argument として理解するのが自然です。

1.4 対話型と非対話型

対話型（interactive）

証明者 P と検証者 V が、複数回メッセージをやり取りします。

- V がランダムなチャレンジを送る
- P がそれに応じて応答する
- これを何ラウンドか繰り返す

という形です。

非対話型（non-interactive）

証明者が**一つの証明文字列**を作り、検証者がそれを単独で検査します。

ブロックチェーンのように「後から誰でも検証したい」場面では非対話型が特に便利です。

Fiat–Shamir 変換

多くの**公開コイン**対話型プロトコルでは、検証者のランダムチャレンジをハッシュ関数で置き換えることで、非対話化できます。これを **Fiat–Shamir 変換** と呼びます。

雑に言えば、

1. 証明者はこれまでの transcript をハッシュする
2. ハッシュ値を「検証者が送ったはずの乱数」とみなす
3. その乱数に対する応答を続ける

という手順です。

この変換は非常に重要ですが、厳密には**ランダムオラクルモデル**などの仮定のもとで議論されることが多いです。

1.5 公開コインと透明性

公開コイン（public-coin）

検証者のメッセージが「ただのランダム文字列」であるタイプのプロトコルです。
検証者が難しい秘密計算をする必要がありません。

Ligero も STARK 系も、この公開コイン性が重要です。
なぜなら、

- 実装が単純になる
- Fiat–Shamir に乗せやすい
- trusted setup を避けやすい

からです。

透明性（transparent）

STARK の文脈で特に強調される言葉で、ざっくり言えば

> **秘密の共通参照文字列や trapdoor を必要としない**

という意味です。

pairing-based SNARK の多くでは、最初に「安全に作られた設定」が必要でした。
これが漏れると偽証が可能になることがあります。
一方、transparent な系では、公開乱数やハッシュ関数を中心に構成するため、その種の setup 依存を避けられます。

1.6 PCP, IOP, IOPP を最小限だけ

Ligero や zk-STARK を読むと、PCP や IOP という言葉が出てきます。

PCP（Probabilistically Checkable Proof）

検証者が証明全体を読まず、**ランダムに少数箇所だけ読んで**高い確率で正しさを判定できる証明の概念です。

IOP（Interactive Oracle Proof）

証明者が大きなオラクル（長い文字列）を提示し、検証者がそこへランダムアクセスしながら、さらに対話も行うモデルです。

IOPP（Interactive Oracle Proof of Proximity）

ある長い文字列が、ある符号や集合に「属している」あるいは「近い」ことを検査するための IOP です。

STARK の FRI は、この IOPP の文脈で理解すると見通しが良くなります。
Ligero もまた、巨大な構造にランダムアクセスしながら矛盾を探す、という意味でこの世界観に属します。

1.7 なぜ「全部見ない」のに信用できるのか

初学者が最も感じる違和感はここでしょう。

理由 1：嘘をつくと、多くの場所で矛盾するように設計されている

たとえば低次数多項式であるはずの表が、実はバラバラな値の寄せ集めなら、ランダムに選んだ少数点でも不整合に当たりやすくなります。

理由 2：ランダム線形結合で、多数の制約を一つに圧縮できる

100 個の制約をそのまま全部見る代わりに、ランダム係数で混ぜた一つの制約を見ます。
もし元の制約群のどれかが壊れていれば、混ぜた結果も高確率で壊れます。

理由 3：コミットメントで後出し変更を防げる

証明者は最初に長い表やオラクルにコミットし、その後でランダム位置を開きます。
Merkle 木は、その「後出しの改ざん防止」に使われます。

この三つの組み合わせが、Ligero と zk-STARK の共通骨格です。

1.8 典型的な誤解

誤解 1：「零知識なら何も情報を出してはいけない」

実際には、「**証明対象が真である**」という事実は当然伝わります。
隠したいのは witness の中身や、そこから得られる追加情報です。

誤解 2：「健全性誤差があるなら危険」

健全性誤差は暗号で普通に使われる概念です。
重要なのは、その値を十分小さくできることです。

誤解 3：「透明なら自動的に軽い」

透明性と軽さは別問題です。
transparent でも証明サイズが大きいことはあります。
実際、STARK は透明ですが、pairing-based SNARK に比べると証明が大きいことが多いです。

誤解 4：「ゼロ知識証明は秘密の開示ゼロなので、計算の痕跡も不要」

実際には、証明者はかなり複雑な内部計算をします。
秘密を出さないために、むしろ**余分な符号化・ブラインディング・ハッシュ**が必要になることも多いです。

1.9 小さな例：パスワードを見せずに知っていることを示す

公開値として $h = H(\text{password})$ が登録されているとします。
ログイン時にサーバへパスワードそのものを送る代わりに、

> 「私は $H(w) = h$ を満たす w を知っている」

とゼロ知識で示せれば、平文パスワードは渡さずに済みます。

もちろん、実際に安全なプロトコルにするには、再利用攻撃や辞書攻撃、セッション束縛など多くの注意が必要です。
しかし「秘密を送らずに条件成立だけを示す」というゼロ知識の感覚は、この例で掴みやすいです。

1.10 この章のまとめ

- ゼロ知識証明では、公開主張 x と秘密 witness w の関係 $R(x, w)$ を扱う。
- 核となる性質は **完全性・健全性・零知識性** の三つである。
- 実用系では proof より argument であることが多い。
- Ligero や zk-STARK を理解するには、**公開コイン**、**Fiat–Shamir**、**オラクルへのランダムアクセス**の感覚が重要である。
- 次章では、これらを支える数学的道具 —— 有限体、多項式、Reed–Solomon 符号、Merkle 木 —— を準備する。

練習問題

1. ハッシュ前像の例で、statement と witness を自分の言葉で書いてください。
2. 「零知識なのに、主張が真だという事実は伝わる」のはなぜ矛盾しないか、説明してください。
3. proof と argument の違いが、どのような安全性の前提の違いに対応するか述べてください。
4. Fiat–Shamir 変換が便利な理由を、ブロックチェーンの利用例を意識して説明してください。

第 2 章 数学と道具の準備

この章では、Ligero と zk-STARK を読むために必要な道具を、必要最小限に絞って導入します。
本格的な代数の教科書ではありません。目的は、後の章で出てくる

- 有限体
- 多項式
- Reed–Solomon 符号
- 低次数性
- Merkle 木
- オラクルとランダム照会

を「読める」状態にすることです。

2.1 有限体とは何か

まずは $\mathbb{F}_p$ だけでよい

最初は、素数 p に対して

$$\mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$$

を「**p で割った余りの世界**」だと思えば十分です。
足し算も掛け算も、最後に p で割った余りを取ります。

たとえば $\mathbb{F}_{17}$ では、

- $15 + 5 = 20 \equiv 3 \pmod{17}$
- $4 \cdot 5 = 20 \equiv 3 \pmod{17}$

です。

なぜ有限体が必要なのか

ゼロ知識証明では、ランダム係数を選んだり、多項式を評価したり、符号語を作ったりします。
そのとき、

- 四則演算がきれいに閉じる
- ゼロでない元には逆元がある
- 多項式の性質が扱いやすい

という理由で、有限体が自然な舞台になります。

STARK では $\mathbb{F}_{2^m}$ もよく出る

STARK 系の文献では、$\mathbb{F}_{2^{64}}$ のような**拡大体**が現れます。
初学者は最初からこれを完全に理解しなくてよいです。
「0/1 のビット列に近い表現を持ちつつ、体として振る舞う大きな有限集合」くらいに見ておけば十分です。

本書の例は多くを $\mathbb{F}_{17}$ や小さな整数例で説明します。

2.2 多項式は “構造を持った関数” である

多項式

$$p(X) = a_0 + a_1 X + a_2 X^2 + \cdots + a_d X^d$$

は、単なる関数ではありません。
次数が低いというだけで、値の並び方に強い制約がかかります。

重要事実 1：次数 d の多項式は $d+1$ 点で決まる

たとえば一次式 $a + bX$ は 2 点、二次式は 3 点の値が分かれば一意に決まります。

これは「少数の情報から関数全体が決まる」という意味で、証明系にとって非常に便利です。

重要事実 2：異なる低次数多項式は、そんなに多くの点で一致できない

もし p と q が次数 $\leq d$ の異なる多項式なら、
$p(X) - q(X)$ は零多項式ではないので、根の数は高々 d 個です。

つまり、

> **異なる低次数多項式は、たまたま少数点で一致しても、多数点で一致し続けることはできない**

のです。

この事実が、ランダム点検査の安全性を支えます。

2.3 補間：値の表から多項式を戻す

有限体上でも、実数と同じように補間ができます。
たとえば $\mathbb{F}_{17}$ で、点

- $p(1) = 2$
- $p(2) = 3$

を満たす一次多項式を探すと、

$$p(X) = 1 + X$$

でよいことが分かります。

なぜなら、

- $p(1) = 1 + 1 = 2$
- $p(2) = 1 + 2 = 3$

だからです。

Ligero では、少数個のメッセージ値から多項式を作り、それを多数点で評価して**符号語**にします。
STARK では、実行トレースの列を多項式として補間し、さらに大きな領域で評価します。

2.4 Reed–Solomon 符号の直観

Reed–Solomon（RS）符号は、ざっくり言えば

短いメッセージを、多項式の多数点評価として冗長に並べたもの

です。

基本形

- 低次数多項式 $p(X)$ を選ぶ
- あらかじめ決めた点 $\eta_1, \eta_2, \dots, \eta_n$ で評価する
- 得られた値列
$$(p(\eta_1), p(\eta_2), \dots, p(\eta_n))$$
を符号語とする

という仕組みです。

小さな例

$\mathbb{F}_{17}$ で

$$p(X) = 1 + X$$

を考え、評価点を $0, 1, 2, 3$ とすると、

$$(1, 2, 3, 4)$$

が符号語です。

この列はただの 4 つ組ではありません。
「**一つの一次多項式から来ている**」という構造を持っています。

なぜ冗長化するのか

元のメッセージが 2 点分の情報しかなくても、4 点や 8 点に広げておけば、

- 一部が壊れていても検出しやすい
- ランダムに数点見るだけで不正を見つけやすい
- Merkle 木でコミットしやすい

といった利点があります。

2.5 “近い” とは何か：距離の考え方

符号理論では、二つの長い列がどれくらい違うかを、しばしば**ハミング距離**で測ります。

ハミング距離

二つの同長ベクトル u, v に対し、違う座標の個数を数えるものです。

たとえば

$$u = (1, 2, 3, 4), \quad v = (1, 9, 3, 8)$$

なら、違うのは第 2 成分と第 4 成分なので距離は 2 です。

“コードに属する” と “コードに近い”

STARK の FRI では、「この長い表は本当に低次数多項式の評価列か？」を直接判定する代わりに、

- コードに属するか
- あるいは、コードから十分遠いか

を見分ける**近接性検査（proximity testing）**を行います。

この “近い” という考え方が、IOPP の中心にあります。

2.6 低次数性がなぜ嬉しいのか

次の一文が非常に重要です。

> **低次数性は、巨大な表に “圧縮された一貫性” を与える。**

たとえば、長さ 1024 の表が「本当に次数 15 の多項式から来ている」と分かれば、その表は 16 個程度の係数で記述できる強い構造を持ちます。

このおかげで、

- 表全体を見なくても
- ランダムな数点と
- いくつかの整合性検査だけで

不正を炙り出せます。

Ligero は「各行が低次数多項式から来る」ことを使います。
zk-STARK は「トレース列や制約多項式が低次数である」ことを使います。

2.7 ランダム点検査の基本原理

低次数多項式の強さを、最も単純な形で言うとこうです。

命題（直観版）

異なる二つの次数 $\leq d$ の多項式 p, q を、大きな集合 S 上で比べるとする。
ランダムな $r \in S$ を選んで $p(r) = q(r)$ が成り立つ確率は、おおよそ $d/|S|$ 以下です。

つまり、評価集合 S が十分大きければ、ランダム一点を見るだけでもかなりの検査力があります。

ゼロ知識証明とのつながり

もちろん実際のプロトコルでは、比較したいものが単純な二つの多項式ではなく、

- 行列の各行
- トレースの各列
- 制約をまとめた composition polynomial

だったりします。
しかし本質は同じで、「**低次数なはずなのにそうでない**」というズレをランダム検査で見つけます。

2.8 Merkle 木：大きな表へのコミットメント

Ligero でも STARK でも、証明者は巨大な表やオラクルを用意します。
しかし検証者はその全体を受け取りません。そこで必要になるのが**コミットメント**です。

Merkle 木は、長いデータ列 $(a_1, \dots, a_n)$ に対し、

1. 各要素をハッシュして葉にする
2. 隣り合うハッシュをさらにハッシュして親を作る
3. これを根まで繰り返す

という二分木です。

最終的な root hash だけを公開すれば、後から任意の要素 a_i を開示するときに、

- その値
- 兄弟ノードのハッシュ列（authentication path）

を示すだけで、その値が最初の root に整合していることを検証できます。

なぜ便利なのか

- **バインディング**：後から都合よく値を差し替えにくい
- **部分開示**：必要な位置だけ見せればよい
- **公開検証**：誰でも root とパスから確認できる

STARK の「大きな評価表にコミットして、あとでランダム位置を開く」という流れは、Merkle 木を抜きに理解できません。

2.9 オラクルモデルとは何か

PCP/IOP の文脈で「オラクル」というと、しばしば

> **長い文字列にランダムアクセスできるブラックボックス**

のことです。

検証者は、その全部を読む代わりに、必要な位置だけ問い合わせます。
たとえば「表の 57 番目と 913 番目を見せてください」と言うわけです。

実際の実装では、これは

- Merkle root にコミットされた大きな配列
- 指定位置の値と認証パスの返答

として実現されます。

2.10 低次数拡張（Low-Degree Extension; LDE）

STARK を理解するうえで重要な語が **LDE** です。

もともと短い領域 H の上に定義された列を、

1. それを補間して多項式 $p(X)$ とみなす
2. より大きな領域 L 上で p を評価する

ことで、より長い評価列に拡張します。

なぜ拡張するのか

拡張すると冗長性が増え、低次数性の検査がしやすくなります。
また、制約の一部を別の点で評価する余地ができ、ランダム検査がやりやすくなります。

STARK の実装では、この LDE が中心的です。
Ligero でも「少数のメッセージ値を多数点評価にする」という意味で、近い発想が見られます。

2.11 Interleaving の直観

Ligero で重要なのが **interleaved Reed–Solomon code** です。
これは難しそうに見えますが、直観は簡単です。

- 1 本の RS 符号語を使う代わりに
- **複数本の RS 符号語を行として並べた行列**を考える

だけです。

たとえば 3 行あるなら、

$$U = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}$$

で、各 u_i がそれぞれ RS 符号語です。

この表現が便利なのは、**行方向のランダム線形結合**を取ると、また 1 本の RS 符号語になるからです。
Ligero は、この性質を使って「多くの行をまとめて一度に検査」します。

2.12 ここまでの道具がどう使われるか

この章の道具が、後でどう現れるかを先にまとめておきます。

道具	Ligero での役割	zk-STARK での役割
有限体	回路値と乱数の計算	トレース、AIR、FRI の舞台
多項式	行の符号化	トレース列・制約・低次数検査
Reed–Solomon 符号	行列の各行を冗長化	FRI の低次数コードの中心
ハミング距離	“コードからどれだけ外れているか” の評価	近接性検査の基本概念
Merkle 木	非対話化・公開検証の支え	巨大オラクルへのコミットメント
ランダム照会	少数の点だけ検査する	少数の位置だけ開示する

2.13 この章のまとめ

- 有限体は、ゼロ知識証明の代数的な舞台である。
- 低次数多項式は、少数の情報で強い構造を持つ。
- Reed–Solomon 符号は、多項式の評価列として自然に作られる。
- ランダム点検査が効くのは、異なる低次数多項式は多くの点で一致できないからである。
- Merkle 木は、大きな表にコミットしつつ一部だけ開示するための道具である。
- Ligero と zk-STARK は、これらの道具を異なる流儀で組み合わせている。

練習問題

1. $\mathbb{F}_{17}$ で $p(X) = 3 + 2X$ を $X = 0, 1, 2, 3$ で評価してください。
2. なぜ次数 d の多項式は $d + 1$ 点で決まるのか、直観的に説明してください。
3. Reed–Solomon 符号が「冗長化」になっている理由を述べてください。
4. Merkle 木で root だけを先に公開することが、なぜ後出し変更の防止につながるのか説明してください。
5. interleaving を「複数の RS 符号語を縦に積むこと」と見たとき、ランダム線形結合がなぜ重要か考えてみてください。

第 3 章 Ligero の全体像

この章では、Ligero を**細部に入る前に**大づかみに理解します。
Ligero の価値は、単に「trusted setup が要らない」ことだけではありません。

原論文が強く打ち出したのは、次の組み合わせです。

- **サブリニア通信**
- **単純な暗号部品**
- **公開コイン**
- **具体的にもそこそこ軽い**
- **複数インスタンスでの償却効果**

まずは「Ligero が何を狙ったプロトコルか」を掴みましょう。

3.1 Ligero とは何か

Ligero は、2017 年の CCS に登場した
"Lightweight Sublinear Arguments Without a Trusted Setup"
という論文に由来する零知識 argument 系です。

ざっくり言うと、Ligero は

> **NP の関係を満たす witness の存在を、回路サイズの平方根に比例する程度の通信で示したい**

という発想に立つプロトコルです。

ここでの「平方根」は非常に重要です。
従来、回路サイズ s に対して証明通信が $O(s)$ 近くになると、大きな回路では厳しい。
Ligero は、適切に構造化すれば、通信を概ね $O(\sqrt{s})$ に近づけられることを示しました。

3.2 どうして平方根が出てくるのか

Ligero の心臓部は、「証明すべき大量のデータ」を**行列**に並べることです。

たとえば総量が s のデータを、

- 行数 m
- 列数 ℓ

の行列に並べて $m\ell \approx s$ とします。

このとき、典型的には

- 行に関するコストが m に比例
- 列を開くコストが ℓ に比例

するような設計になります。

すると総コストは大雑把に

$$m + \ell$$

の形になります。
ここで $m\ell \approx s$ を保ったまま $m + \ell$ を最小にするには、
バランスのよい

$$m \approx \ell \approx \sqrt{s}$$

を選べばよい。

これが、Ligero の「平方根」直観です。

大事な注意

これは**本質を掴むための簡略化**です。
実際の定数や追加項はもっと複雑で、符号長、繰り返し回数、健全性誤差、体サイズなどに依存します。
しかし初学者にとっては、まずこの「**二方向のコストの釣り合い**」が見えていることが重要です。

3.3 Ligero の設計思想

Ligero の設計思想を四つに分けると理解しやすいです。

1. witness と回路制約を “表” にする

証明したいのは「ある witness が回路を満たす」という事実です。
Ligero はこれを、行列やベクトルの整合性チェックに落とします。

2. 各行を Reed–Solomon 符号で冗長化する

行の値そのものを渡す代わりに、各行を**低次数多項式の評価列**として表現します。
これにより、「ちゃんとした行かどうか」を少数の検査で見やすくなります。

3. ランダム線形結合で多数の条件を一つに縮約する

各行を一個ずつ検査すると重いので、検証者はランダム係数を選び、
それらの線形結合だけを見ます。
壊れた行がどこかにあれば、高確率で線形結合にも矛盾が現れます。

4. 少数の列だけを開かせる

証明者に表の全部を見せさせるのではなく、ランダムに選んだ少数列だけを開かせます。
これにより、通信が大きくならないようにします。

3.4 Ligero の全体フロー

細部を省いて全体像だけ描くと、Ligero は次のように見えます。

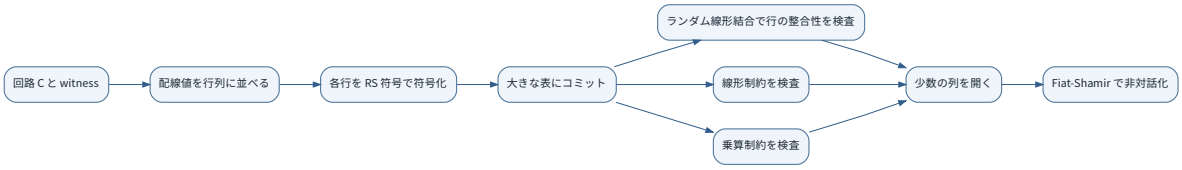

図 2: Ligero の高レベルな証明フロー

このフローから分かるように、Ligero の主要部品は

- **RS 符号**
- **ランダム線形結合**
- **少数列の開示**
- **ハッシュによるコミットメント**

です。

pairing や楕円曲線ペアリングのような重い公開鍵暗号を中核に置かないところが、Ligero の特徴です。

3.5 何が嬉しいのか

Ligero の魅力は、理論と実装の中間にあります。

trusted setup が不要

pairing-based SNARK の多くでは、危険な setup が必要でした。
Ligero は、その類の trapdoor 依存を避ける方向にあります。

使う部品が比較的単純

衝突耐性ハッシュ関数や、有限体上の演算、符号理論が中心です。
そのため、設計思想が追いやすく、量子計算への耐性を議論しやすい立場にあります。

通信がサブリニア

証明対象が十分大きいとき、証明者が表全体を送る必要がなく、回路サイズよりかなり小さい通信で済みます。

複数インスタンスの償却が効く

原論文では、多数の同型インスタンスをまとめて扱うと、一件あたりの通信がさらに下がることが示されています。
これは「同じ回路で多くの証明をまとめたい」状況で魅力的です。

3.6 原論文の代表的な結果をどう読むか

Ligero の原論文では、SHA-256 前像を例に、かなり具体的な性能評価が示されました。
有名なのは次のような主張です。

- 健全性誤差 2^{-40} の設定で、SHA-256 前像のゼロ知識証明に対し、およそ **44KB** 程度の通信
- 特定の仮定のもとではそれより短い通信
- 多数インスタンスでは、1 件あたり通信をさらに償却できる

ここで重要なのは、数値そのもの以上に、

> **“透明性寄りの系で、具体値としても十分戦える”**

と示した点です。

ただし、これを今日のあらゆる証明系と単純比較してはいけません。
証明サイズ、証明時間、検証時間は、

- 回路の種類
- 体の選び方
- 実装言語
- 健全性誤差

- 非対話化の方法
- バッチの有無

で大きく変わるからです。

3.7 Ligero はどんな問題に向くか

Ligero は、ざっくり言うと次のような場面に向いています。

1. 中規模〜大規模の回路を、trusted setup なしで証明したい

たとえば

- ハッシュ計算の正しさ
- 認証・前像知識
- 回路として表しやすい検査

などです。

2. 複数の同種インスタンスをまとめたい

たとえば多数の証明要求が同じ回路を共有する場合、償却の恩恵が得やすいです。

3. 重い公開鍵暗号ベースの SNARK を避けたい

システム上の理由や安全性の好みから、ハッシュ・符号ベースの構成を好む場合に選択肢になります。

3.8 Ligero が万能ではない理由

Ligero を過大評価しないために、弱みも先に挙げておきます。

超巨大計算では STARK 的発想が強いことが多い

非常に長い逐次計算や VM 実行全体を証明するなら、トレースベースの STARK が自然なことが多いです。

証明サイズが pairings 系 SNARK より小さいとは限らない

「trusted setup は嫌だが、証明は極小がよい」という要件では、別系統の方が有利なことがあります。

数式の見通しはよいが、実装は依然として簡単ではない

RS 符号・行列配置・制約圧縮・ハッシュコミットメントを誤りなく組み合わせる必要があるため、研究実装としては十分に難しいです。

3.9 Ligero を一文で言うなら

Ligero を一文で言うなら、私は次のように要約します。

> **「回路満足性を、行列化された低次数構造のランダム検査に落とし、平方根オーダの通信で示す、透明寄りの零知識 argument」**

この一文の中に、

- 回路
- 行列
- 低次数

- ランダム検査
- 平方根
- 透明性

というキーワードが全部入っています。
以後の章では、この一つ一つを分解します。

3.10 この章のまとめ

- Ligero は、回路サイズの平方根に比例する通信を狙う零知識 argument 系である。
- 核心は、**行列配置・RS 符号・ランダム線形結合・少数列開示**である。
- 「平方根」の直観は、行方向と列方向のコストを釣り合わせることから出てくる。
- trusted setup を避けつつ、具体性能でも意味のある領域を狙った点が Ligero の大きな価値である。
- 次章では、行列・interleaved RS 符号・線形制約検査・乗算制約検査という内部構造を詳しく見る。

練習問題

1. Ligero の通信量に平方根が現れる直観を、自分の言葉で説明してください。
2. 「各行を RS 符号で冗長化する」とは、後でどのような検査を可能にするためか考えてみてください。
3. Ligero の利点と欠点を、それぞれ 2 つずつ挙げてください。
4. 同じ “transparent な証明系” でも、Ligero と STARK が異なる思想を持つことを一文で表現してみてください。

第 4 章 Ligero の中身を分解する

ここからは、Ligero の内部をもう少し厳密に見ます。
ただし原論文そのままの記法を全部追うのではなく、理解の流れを優先して

1. 何を行列に入れるのか
2. interleaved Reed–Solomon 符号とは何か
3. 行が正しい符号語であることをどう検査するか
4. 回路の線形制約・乗算制約をどう検査するか
5. どうやって零知識・非対話化を得るか

の順で説明します。

4.1 出発点：回路満足性

Ligero が最終的に示したいのは、

> **ある算術回路 C に対して、秘密 witness w を入れると $C(w) = 1$ である**

という事実です。

なぜ算術回路なのか

ビット演算や論理回路も最終的には多くの場合、有限体上の算術制約に変換して扱えます。
たとえばブール値 $b \in \{0, 1\}$ は

$$b(b-1) = 0$$

という式で表せます。
AND, XOR, 加算器なども有限体上の式に落とせます。

そのため、まずは「加算ゲートと乗算ゲートからなる算術回路」として理解するのがよいです。

4.2 witness と配線値を行列に並べる

回路の各ワイヤに入る値をすべて集めると、長いベクトルになります。
Ligero はこの長いベクトルを、適当な行数 m と列数 ℓ を使って行列に並べます。

$$M \in \mathbb{F}^{m \times \ell}$$

としましょう。

この行列化の意味は二つあります。

意味 1：巨大な witness を二次元にする

一次元の長い列のままだと、「どこをどう開くか」「何をまとめて検査するか」が扱いにくいです。
二次元にすると、

- 行方向にまとめて圧縮する
- 列方向に spot-check する

という役割分担が自然になります。

意味 2：コスト最適化に使える

第 3 章で見たように、行数と列数をうまく選ぶと、
証明に必要な通信を $m + \ell$ 型でバランスできます。

4.3 各行を RS 符号で符号化する

ここが Ligero の代数的なコアです。

各行 M_i を、少数点のメッセージから**低次数多項式** $p_i(X)$ として補間し、
より多くの評価点 $\eta_1, \dots, \eta_n$ で評価して、長い符号語

$$U_i = (p_i(\eta_1), \dots, p_i(\eta_n))$$

にします。

行全体を並べると、

$$U = \begin{pmatrix} U_1 \\ \vdots \\ U_m \end{pmatrix}$$

という $m \times n$ 行列になります。
各行が RS 符号語であるので、これは **interleaved Reed–Solomon code** の語だと考えられます。

interleaved とは何か

難しく見えますが、意味は単純です。

> **“複数本の RS 符号語を、行として縦に積んだもの”**

です。

なぜこの表現が強いのか

RS 符号は線形コードなので、行に対してランダムな線形結合を取ると、また RS 符号語になります。
つまり、検証者がランダム係数 $r_1, \dots, r_m$ を選び、

$$w = \sum_{i=1}^{m} r_i U_i$$

を考えると、正直な場合には w も RS 符号語です。

この一事が、Ligero の検査を劇的に軽くします。

4.4 行の “符号語らしさ” をどう検査するか

もし証明者が勝手な行列 U を出しても、各行が本当に RS 符号語かどうかを全部読むのは重すぎます。
そこで Ligero は、次のような検査をします。

membership test の直観

1. 検証者はランダム係数ベクトル $r \in \mathbb{F}^m$ を選ぶ

2. 証明者は線形結合
$$w = r^T U$$
を返す
3. 検証者は、w が RS 符号語であることを確認する
4. さらに、ランダムに少数列 j を選び、その列で
$$w_j = \sum_i r_i U_{i,j}$$
が成り立つことを確認する

これでなぜ十分なのか

もし多数の行が怪しいなら、そのランダム線形結合 w も高確率で怪しくなります。
一方、証明者が都合よく w を別物として偽装しようとしても、後でランダム列を開かせれば整合性が崩れやすい。

要するに、

- **行方向**はランダム線形結合で圧縮し
- **列方向**はランダム列開示で整合性を見る

という二段構えです。

4.5 なぜランダム線形結合が効くのか

Ligero の核心を一行で言えばこれです。

> **多数の条件を、ランダムに混ぜた一つの条件として検査する。**

この発想は、後で STARK の ALI や composition polynomial にも再び現れます。

一般原理

もしベクトル $v_1, \ldots, v_m$ のどれかが壊れているなら、
ランダム係数 r_i に対する和 $\sum_i r_i v_i$ も、高確率で壊れたままです。

もちろん「たまたま打ち消し合う」可能性はあります。
しかし係数を大きな有限体からランダムに選べば、その確率は十分小さくできます。

初学者向けの比喩

100 枚の答案のうち数枚が改ざんされているとして、各答案をランダム係数で混ぜた “合成答案” を作るとします。
改ざん箇所を正確に相殺するのは難しい。
だから合成答案の検査で、不正が表面化しやすい。
これがランダム線形結合の感覚です。

4.6 線形制約の検査

回路の加算ゲートやコピー制約は、**線形制約**として表せます。
たとえば

$$z = x + y$$

は

$$x + y - z = 0$$

という一次式です。

多くの線形制約をまとめると、行列表現で

$$Ax = b$$

の形になります。

Ligero のやり方

Ligero は、この $Ax = b$ を各制約ごとに別々に見るのではなく、
やはりランダム係数で混ぜた一つの制約として見ます。

1. 検証者はランダムベクトル r を選ぶ
2. 証明者は $(r^T A)x$ を表す多項式の係数を送る
3. 検証者は
   - その多項式が $r^T b$ と整合すること
   - もとの RS 符号化された表と整合すること を、少数の点で確認する

直観

全部の線形制約を読む代わりに、「ランダムな重み付き和が 0 であること」を見ます。
どれかの制約が壊れていれば、その重み付き和も高確率で壊れます。

4.7 乗算制約の検査

回路の本質的な難しさは、線形ではなく**乗算**です。
たとえばゲート

$$z = x \cdot y$$

は二次制約です。

Ligero では、x, y, z に対応する符号化表を用意し、
それらが

$$x \circ y = z$$

（$\circ$ は成分ごとの積）を満たすかを検査します。

基本アイデア

- x, y, z の各行を表す多項式を考える
- ランダム線形結合を取る
- 積の関係を満たすはずの多項式 p_0 を証明者に送らせる
- ランダム列で実際の値と整合しているかを見る

なぜ難しいのか

線形制約なら線形結合がそのまま使えますが、積が入ると次数が上がります。
そのため、検査対象の多項式の次数や符号が少し変わります。
原論文では、この点を丁寧に扱って soundness を示しています。

初学者向けの見方

「乗算ゲートが正しいか」を全部見る代わりに、
多くの乗算ゲートをランダム係数で混ぜた一つの “合成乗算チェック” を作り、
それが正しいはずの多項式になるかを少数点で検査する、と思えばよいです。

4.8 回路全体はどう処理されるか

結局、算術回路は

- 入力・出力の整合性
- コピーや加算などの線形制約
- 乗算ゲートの二次制約

の組み合わせです。
Ligero はこれらをすべて、RS 符号化された行列の上の検査に落とします。

この観点では、Ligero は

> 「**回路を符号化された表に写し、表がいくつかの代数的性質を持つことを示すプロトコル**」

と言えます。

4.9 零知識はどう入るのか

ここまでの説明だけだと、「ランダム列を開かせたら witness が漏れそうだ」と思うはずです。
その通りで、**そのままでは零知識になりません**。

Ligero の零知識性は、もともと MPC-in-the-head / IKOS 系の変換に由来する視点から理解すると分かりやすいです。

非形式的な説明

- witness をそのまま見せるのではなく、マスクされた形・共有された形で扱う
- 検証者が開かせるのは、その一部の整合したビューだけにする
- それでも全体の整合性は高確率で保証されるよう設計する

原論文では、基礎にある MPC プロトコルのプライバシーから零知識性を導いています。

初学者が覚えるべき点

零知識のためには、単に「少ししか開かない」だけでは足りません。
開かれる少数の情報から witness が推測できないように、事前にランダム化されていることが必要です。

4.10 コミットメントと非対話化

対話型の Ligero では、検証者が乱数を送り、それに応じて証明者が返答します。
これを非対話型にしたいときは、

- 各オラクルや表にハッシュでコミットし
- チャレンジを transcript から Fiat–Shamir で導き
- 必要な位置だけ Merkle path 付きで開く

という形を取ります。

ここでハッシュが重要になる理由

- 証明者は先に表を固定しなければならない
- あとからチャレンジを見て都合よく内容を変えてはいけない
- そのため、コミットメントが必要
- Merkle 木は巨大な表に対する効率的なコミットメントになる

proof ではなく argument になる理由

この段階で安全性は、通常

- ハッシュの衝突耐性
- Fiat–Shamir の安全性仮定（ランダムオラクル等）

に依存します。
したがって実用形は argument と呼ぶのが自然です。

4.11 Ligero の内部を一枚絵にすると

この図で重要なのは、「回路を直接検査している」のではなく、
符号化された表の整合性を検査しているという点です。

4.12 ありがちな疑問

Q1. なぜ列ではなく行を RS 符号化するのか

行列の持ち方自体は設計パラメータですが、Ligero では「行方向のランダム線形結合」と「列方向の局所開示」の役割分担が自然に働く配置を使います。

Q2. すべての不正は本当に見つかるのか

確率 1 ではありません。
ただし、不正な表がコードから遠いなら、ランダム線形結合とランダム列開示で、健全性誤差を十分小さくできます。

Q3. では witness を少しずつ推測されないか

零知識は、単なる spot-check のおかげではなく、**マスキングや MPC 的なプライバシーの設計**も含めて成り立ちます。

4.13 この章のまとめ

- Ligero は、回路満足性を符号化された行列の整合性検査へ落とす。
- 各行を RS 符号化した行列が **interleaved Reed–Solomon code** の語になる。
- ランダム線形結合により、多数の行や多数の制約を一つの検査に圧縮できる。

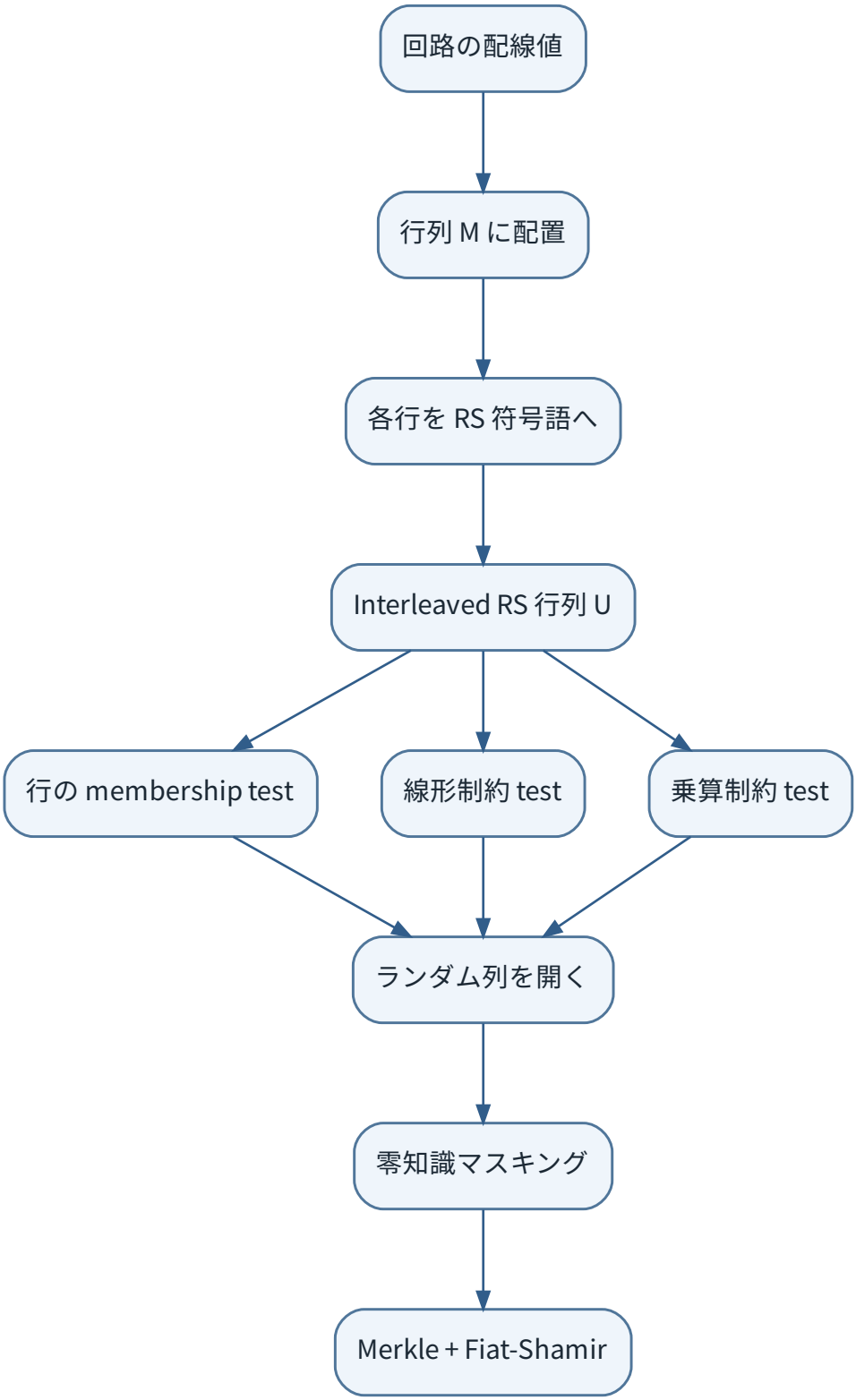

図 3: Ligero の内部構造と検査項目

- 線形制約と乗算制約は、それぞれ対応する多項式検査として扱われる。
- 零知識は、開示量が少ないことだけでなく、マスキングや MPC 的プライバシー設計によって支えられる。
- 非対話化にはハッシュコミットメント、Merkle 木、Fiat–Shamir が重要である。

練習問題

1. interleaved RS 符号を「複数本の RS 符号語を並べた行列」と理解したとき、ランダム線形結合の意味を説明してください。
2. 線形制約 $Ax = b$ を、なぜランダム線形結合で一つにまとめられるのか説明してください。
3. 乗算制約が線形制約より難しいのはなぜか、次数の観点から述べてください。
4. Merkle 木が Ligero の非対話化に必要な理由を説明してください。
5.「少数列しか開かない」だけでは零知識性が保証されない理由を考えてください。

第 5 章 Ligero を小さな回路で追う

この章では、Ligero を**小さな例**で追います。
本物のパラメータや soundness 解析はもっと複雑ですが、
「何がどこで検査されているのか」を掴むには、小例が最も有効です。

5.1 証明したい主張

次の公開主張を考えます。

> **公開値 $z = 8$ に対し、秘密値 a, b が存在して**
>
> $$a \cdot b + a = z$$
>
> **を満たす。**

たとえば witness として

$$a = 2, \quad b = 3$$

を持っていれば、

$$2 \cdot 3 + 2 = 8$$

なので主張は真です。

5.2 回路に直す

この関係を小さな算術回路にすると、内部配線値はこうなります。

1. 乗算ゲート

$$g_1 = a \cdot b$$

2. 加算ゲート

$$g_2 = g_1 + a$$

3. 出力制約

$$g_2 = z$$

witness を含めた配線値は

$$(a, b, g_1, g_2) = (2, 3, 6, 8)$$

です。

5.3 行列に並べる

簡単のため、これを 2×2 行列に並べます。

$$M = \begin{pmatrix} a & b \\ g_1 & g_2 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 6 & 8 \end{pmatrix}$$

本物の Ligero では、もっと大きな行列を使い、どの値をどこに置くかも細かく調整します。
ここでは「行列化して、行を符号化して、列を spot-check する」という構図だけを見ます。

5.4 各行を多項式として見る

有限体 $\mathbb{F}_{17}$ 上で考えましょう。
各行は 2 点のメッセージなので、一次多項式で補間できます。

第 1 行 $(2, 3)$

評価点を $\zeta_1 = 1, \zeta_2 = 2$ とすると、

$$p_1(1) = 2, \quad p_1(2) = 3$$

を満たす一次式は

$$p_1(X) = 1 + X$$

です。

第 2 行 $(6, 8)$

同様に、

$$p_2(1) = 6, \quad p_2(2) = 8$$

を満たす一次式は

$$p_2(X) = 4 + 2X$$

です。

5.5 RS 符号語にする

今度は評価点集合を $\eta = \{0, 1, 2, 3\}$ に広げて、各多項式を評価します。

第 1 行の符号語

$$p_1(0) = 1,\ p_1(1) = 2,\ p_1(2) = 3,\ p_1(3) = 4$$

なので

$$U_1 = (1, 2, 3, 4)$$

です。

第 2 行の符号語

$$p_2(0) = 4,\ p_2(1) = 6,\ p_2(2) = 8,\ p_2(3) = 10$$

なので

$$U_2 = (4, 6, 8, 10)$$

です。

したがって、符号化後の行列は

$$U = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 6 & 8 & 10 \end{pmatrix}$$

になります。

この時点で、元の 2 列の情報が 4 列に増え、冗長性が入っています。

5.6 行の membership test をやってみる

ここで検証者は、「各行が本当に RS 符号語なのか」を全部読む代わりに、
ランダム線形結合で検査します。

ランダム係数を選ぶ

たとえば検証者が

$$r = (5, 7)$$

を選んだとします。

線形結合を作る

すると

$$w = 5U_1 + 7U_2$$

です。成分ごとに計算すると、

$$\begin{aligned} w &= (5 \cdot 1 + 7 \cdot 4,\ 5 \cdot 2 + 7 \cdot 6,\ 5 \cdot 3 + 7 \cdot 8,\ 5 \cdot 4 + 7 \cdot 10) \\ &= (33, 52, 71, 90) \\ &\equiv (16, 1, 3, 5) \pmod{17} \end{aligned}$$

です。

これは本当に RS 符号語か

はい。
なぜなら

$$q(X) = 5p_1(X) + 7p_2(X)$$

とおけば、q も一次多項式であり、

$$\begin{aligned} q(X) &= 5(1+X) + 7(4+2X) \\ &= 33 + 19X \\ &\equiv 16 + 2X \pmod{17} \end{aligned}$$

となるからです。

実際、

- $q(0) = 16$
- $q(1) = 1$
- $q(2) = 3$
- $q(3) = 5$

で、確かに $w = (16, 1, 3, 5)$ になります。

ランダム列を開かせる

検証者は、たとえば評価点 $\eta = 1$（第 2 列）だけ開かせてもよいです。
そのとき元の値は

- 第 1 行：2
- 第 2 行：6

なので、

$$5 \cdot 2 + 7 \cdot 6 = 52 \equiv 1 \pmod{17}$$

であり、確かに w の第 2 成分と一致します。

同様に数列だけ spot-check して整合性を確認します。

5.7 もし不正な行が混じっていたら？

たとえば証明者が、2 行目を勝手に

$$(4, 6, 9, 10)$$

のように改ざんしたとします。
これは一次多項式の評価列ではありません。
なぜなら $(0, 1, 2, 3)$ 上で一次式なら、差分は一定になるはずなのに、

- $6 - 4 = 2$
- $9 - 6 = 3$
- $10 - 9 = 1$

でバラバラだからです。

このときランダム線形結合を取ると、高確率で w も低次数らしさを失います。
さらに、証明者が w だけ都合よく “低次数らしく” 偽装しても、ランダム列を開かせれば元の表と一致しなくなりやすい。

ここに Ligero の健全性の直観があります。

5.8 線形制約をこの例で見る

この回路の線形制約は、

1. 加算ゲート

$$g_2 - g_1 - a = 0$$

2. 出力制約

$$g_2 - z = 0$$

などです。

witness ベクトル

$$x = (a, b, g_1, g_2)$$

を使えば、これらは $Ax = b$ の形にまとめられます。

Ligero は何をするか

各線形制約を一つずつ確認する代わりに、検証者はランダム係数でまとめて

$$r^T(Ax - b) = 0$$

だけを見ます。
証明者は、その値を表す多項式を送り、
検証者は少数の点で元の符号化表と整合するか確認します。

この例での直観

たとえば加算制約だけを見ると、

$$g_2 - g_1 - a = 8 - 6 - 2 = 0$$

です。
もし証明者が g_2 を 9 に偽装していれば、

$$9 - 6 - 2 = 1 \neq 0$$

になります。
複数の線形制約をまとめてランダム重み付き和にしても、壊れた制約があれば高確率でゼロになりません。

5.9 乗算制約をこの例で見る

乗算制約は

$$g_1 = a \cdot b$$

です。
実際には

$$6 = 2 \cdot 3$$

なので正しい。

Ligero の考え方

a, b, g_1 に対応する符号化された値列があるとして、

$$a \circ b - g_1 = 0$$

を満たすことを検査したい。
ここで $\circ$ は成分ごとの積です。

しかし積が入るので、ただの線形関係ではありません。
そこで証明者は、「この積の関係が成り立つならこうなるはず」という**別の多項式**を送り、
検証者はその多項式の値と、ランダム列で開いた実値との整合を確認します。

この例での直観

もし証明者が $g_1 = 5$ と嘘をつけば、

$$a \cdot b - g_1 = 2 \cdot 3 - 5 = 1 \neq 0$$

です。
多数の乗算制約がある場合でも、それらをランダム係数でまとめた多項式検査に落とせます。

5.10 この小例で見えたもの

この例は小さすぎて、Ligero の本当のスケーリングは見えません。
しかし、次の重要な骨格は見えました。

1. “元の witness” ではなく “符号化後の表” を扱う

検査の対象は、もはや (a, b, g_1, g_2) そのものではありません。
それらを符号化した RS 評価表です。

2. ランダム線形結合が中心

行の membership test でも、線形制約でも、乗算制約でも、
「たくさんのものをランダムに混ぜて一つにする」という考え方が出てきます。

3. spot-check は列方向に行う

全部の列を開かず、ランダムな少数列だけを開く。
この一点が、通信量削減の本質です。

5.11 では平方根オーダはどこから来るか

この小例では 2 行 2 列なので実感しにくいですが、
一般に s 個のデータを $m \times \ell$ 行列にして、

- 行方向にまとめるコスト $\sim m$
- 列方向に開くコスト $\sim \ell$

なら、$m\ell \approx s$ のもとで $m + \ell$ を最小化することで

$$m \approx \ell \approx \sqrt{s}$$

が現れます。

この「**行方向圧縮 + 列方向 spot-check**」の設計が、Ligero の平方根感を生みます。

5.12 ブール回路との関係

実際の応用では、SHA-256 のように**ブール回路**として自然な問題も多いです。
Ligero の原論文でも SHA-256 前像が代表例でした。

ブール回路を扱うには、

- ビット制約 $b(b-1) = 0$
- 論理ゲートを算術制約に変換

といったステップが必要です。

後続研究では、Boolean 向けの改良として **BooLigero** や **Ligero++** なども提案されています。
ただし、本書では骨格理解を優先し、算術回路の視点で進めます。

5.13 この章のまとめ

- 小さな回路でも、Ligero の流れは **回路 → 行列 → RS 符号化 → ランダム線形結合 → ランダム列開示** と整理できる。
- 行の membership test では、ランダム線形結合 $w = r^T U$ が中心的役割を果たす。
- 線形制約も乗算制約も、多数の条件をランダムに混ぜた多項式検査として扱われる。
- ここでの「行方向圧縮」と「列方向 spot-check」が、一般のサイズでは平方根オーダの通信へつながる。

練習問題

1. この章の例で、第 1 行 $(2, 3)$ に対応する一次多項式を自分で補間してください。
2. $r = (1, 4)$ を選んだ場合、$w = U_1 + 4U_2$ を計算してください。
3. 2 行目を $(4, 6, 9, 10)$ に改ざんすると一次式の評価列でなくなる理由を説明してください。
4. 線形制約と乗算制約で、証明者が送る多項式の役割の違いを説明してください。
5. 小例では見えにくい平方根オーダの直観を、一般の $m \times \ell$ 行列で説明してください。

第 6 章 zk-STARK の全体像

この章から、もう一つの主役である **zk-STARK** に入ります。
Ligero が「回路制約を符号化表の整合性に落とす」世界なら、
zk-STARK は「**長い計算全体を実行トレースとして捉え、それが局所遷移規則に従うことを示す**」世界です。

この違いを最初に強く意識しておくと、STARK の専門用語が整理しやすくなります。

6.1 STARK という名前

STARK は一般に

- **Scalable**
- **Transparent**
- **ARgument of Knowledge**

の頭文字として説明されます。

文脈によっては「zk-STARK」と言って、**zero-knowledge** 版を明示します。
本書では簡単のため、基本的に「STARK」と言ったら zero-knowledge 版を含む広い意味で使います。

各語の意味

- **Scalable**
  証明対象が大きくなっても、検証コストや証明サイズが比較的穏やかに増えること。
- **Transparent**
  秘密の trusted setup や trapdoor を要しないこと。
- **Argument of Knowledge**
  偽の主張で検証者を騙すには、実際には対応する witness を “持っているに等しい” ことを要求するタイプの argument であること。

6.2 STARK が狙う問題

STARK の思想が特に映えるのは、

> **長くて複雑な計算を、全部やり直さずに検証したい**

という状況です。

典型例

- ブロックチェーン上の大量取引の状態更新
- 大規模な逐次計算
- 巨大なデータベースに対する検索・検証
- 仮想マシン実行の正しさ
- あるアルゴリズムの実行過程全体の正しさ

Ligero も一般計算を扱えますが、STARK は特に
“実行の長さ” が大きい問題に向いた見方を持っています。

6.3 STARK の基本直観

STARK の核心を一文で言うと、

> **計算を「時刻ごとの状態の列」と見なし、その列が低次数多項式として一貫していることを示す**

です。

実行トレース

計算を 1 ステップずつ進めたときの内部状態を表にしたものです。
行が時刻、列がレジスタや変数だと思えばよいです。

AIR

各ステップで満たすべき局所遷移規則を、有限体上の多項式制約として書いたものです。
たとえば

- 次の時刻の第 1 レジスタは、現在の第 2 レジスタと等しい
- 次の時刻の第 2 レジスタは、現在の二つの和である

といった規則を、多項式が 0 になる条件として書きます。

FRI

巨大な評価表が「本当に低次数多項式から来ているか」を、
少数の問い合わせで検査する仕組みです。

Merkle 木

巨大な評価表に対するコミットメントを与えます。

これらが合わさって STARK になります。

6.4 STARK の全体フロー

まずは細部を気にせず、一本の流れとして眺めましょう。

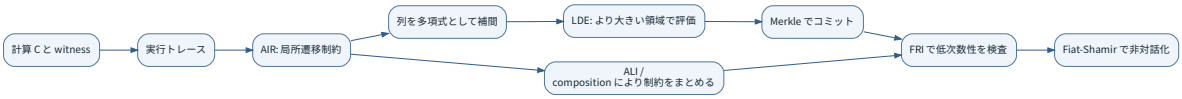

図 4: zk-STARK の高レベルな証明フロー

Ligero と比べると、

- Ligero は「**行列の行**」が主役
- STARK は「**実行トレースの列**」が主役

という違いが見えます。

6.5 透明性とは何か

STARK の売り文句として最も有名なのが **transparent** です。
これは単に「公開されている」という意味ではなく、

> **秘密の setup、trapdoor、特権的な初期生成手順を前提にしない**

という技術的な意味を持ちます。

なぜ重要か

もしシステムが秘密の setup に依存し、その秘密が漏れると偽証が可能になるなら、
その setup 手順自体が社会的・制度的な弱点になります。

一方、transparent な系では、

- 公開乱数
- ハッシュ関数
- 符号理論

を中心に構成するため、その種の信頼集中を弱めやすい。

ブロックチェーンや公共的検証の文脈では、この点が特に強く評価されます。

6.6 “post-quantum secure” と言われる理由

STARK の原論文は、自らを **post-quantum secure** と位置づけています。
ただし、この表現は丁寧に読む必要があります。

どういう意味でそう言うのか

STARK の安全性の中核は、主に

- ハッシュ関数
- 公開乱数
- 符号理論
- ランダムオラクル的仮定

にあります。
少なくとも pairings や離散対数に強く依存する系に比べると、量子計算に対して “好ましい” 立場にある、という主張です。

何に注意すべきか

- 実際の安全性は使うハッシュ関数に依存する
- ランダムオラクルモデルの取り扱いには仮定がある
- 「絶対に量子安全」と軽々しく言うべきではない

したがって本書では、**“量子耐性志向”** あるいは **“量子計算に比較的親和的な仮定に立つ”** というニュアンスで理解することを勧めます。

6.7 STARK の代表的な長所

1. 大規模計算に強い

長い計算をトレースに変換し、局所遷移規則だけを見ればよいので、
逐次計算の証明と相性がよいです。

2. 透明性が高い

trusted setup を避けられることは、理論面でも実運用面でも大きな魅力です。

3. 検証が軽い

プロトコルの設計目標として、検証者は巨大な計算全体をやり直さずに済みます。
この性質が、L2 や公開検証に向きます。

4. 実運用の実績がある

現在のブロックチェーン領域では、STARK 系は単なる理論にとどまらず、
ロールアップやスケーリング基盤として実運用されています。

6.8 STARK の弱み・難しさ

1. 証明サイズが小さいとは限らない

多くの場合、pairing-based SNARK の証明より STARK 証明は大きいです。
透明性の代償の一つとみなせます。

2. 証明者はかなり重い

トレース生成、LDE、Merkle、FRI など、証明者が行う計算は多いです。

3. 概念の層が厚い

AIR、LDE、composition、FRI、Merkle、Fiat–Shamir など、多数の層が重なります。
初学者には Ligero よりも全体像が掴みにくいことがあります。

4. 実装は “理論どおり” ではないことも多い

現代の STARK 実装では、原論文の説明からさらに多くの最適化や派生手法が入っています。
本書ではまず、原論文系の骨格に集中します。

6.9 原論文の象徴的な例：DNA プロファイル照合

STARK の原論文で印象的なのは、
DNA プロファイルが巨大データベースに含まれないことを、データベース自体を公開せずに示す
という例です。

この例が示しているのは、STARK が単なる暗号マニア向けのおもちゃではなく、

- プライバシー
- 公共的検証
- 大規模データ処理

を同時に扱える、という方向性です。

本書では実装の詳細までは追いませんが、「STARK の社会的イメージ」を掴むにはよい題材です。

6.10 Ligero との第一印象の違い

この段階で、Ligero と STARK の違いを再度直観的に言うと、

- **Ligero**
  回路の満足性を、符号化された行列の spot-check に落とす
- **STARK**
  計算の実行過程を、低次数多項式の世界に持ち込み、トレース全体の一貫性を示す

です。

どちらもランダム化と符号理論を使いますが、
Ligero では「行列の表」を見ている感覚が強く、
STARK では「時間発展する計算」を見ている感覚が強い。

6.11 この章のまとめ

- STARK は、**大きな計算をトレースとして捉え、その正しさを低次数多項式の検査に落とす** 証明系である。
- “Scalable” は、大きな計算でも検証が相対的に軽いことを目指す。
- “Transparent” は、trusted setup や trapdoor に依存しないことを意味する。
- STARK の骨格は、**実行トレース → AIR → LDE → Merkle → FRI** で理解するとよい。
- 次章では、STARK の最初の要となる **AIR**、すなわち「計算を状態遷移の多項式制約として書く」部分を詳しく見る。

練習問題

1. STARK の “transparent” が、単に「公開されている」という意味ではないことを説明してください。
2. Ligero と STARK の違いを、「何を圧縮しているか」という観点から述べてください。
3. 実行トレースとは何かを、アルゴリズムの実行履歴の比喩を使って説明してください。
4. STARK の長所と短所を、それぞれ二つずつ挙げてください。

第 7 章 AIR：計算を状態遷移として書く

STARK を理解する最大の山場の一つが **AIR** です。
AIR は **Algebraic Intermediate Representation** の略で、
計算を「**有限体上の状態遷移多項式系**」として書き直したものです。

言い換えると、

> **プログラムの実行が正しいとは、各ステップで局所的な遷移規則が満たされることだ**

という見方を、代数化したものです。

7.1 なぜ AIR が必要なのか

ゼロ知識証明でプログラム全体を扱うには、
最終的に「有限体上の多項式が 0 になるか」という形へ持ち込みたい。
しかし普通のプログラムは、そのままでは

- if 文
- ループ
- メモリアクセス
- レジスタ更新

などから成り、代数的ではありません。

そこでまずやることは、

1. 計算を時系列の**状態列**として表す
2. 1 ステップ先への遷移規則を書く
3. その規則を多項式制約にする

ことです。
これが AIR です。

7.2 実行トレースという見方

たとえばある計算が、各時刻で 2 つのレジスタを持つとします。
すると実行は次のような表で表せます。

時刻 t	レジスタ 1	レジスタ 2
0	a_0	b_0
1	a_1	b_1
2	a_2	b_2
$\vdots$	$\vdots$	$\vdots$

この表を **execution trace** と呼びます。

行と列の意味

- **行**：時間ステップ
- **列**：あるレジスタが時間とともに取る値

STARK は、このトレースを直接見るのではなく、
各列を多項式として表し、さらにその多項式が局所制約を満たすことを示します。

7.3 フィボナッチ列で AIR を作る

最も有名な教材例の一つがフィボナッチ列です。
状態を 2 レジスタ

$$(a_t, b_t)$$

とし、遷移を

$$(a_{t+1}, b_{t+1}) = (b_t,\ a_t + b_t)$$

で定めます。

初期値を

$$(a_0, b_0) = (1, 1)$$

とすると、トレースはこうなります。

t	a_t	b_t
0	1	1
1	1	2
2	2	3
3	3	5
4	5	8

何を証明したいか

たとえば「4 ステップ後に $b_4 = 8$ になる」ということを、
途中の全計算を全部見せずに示したい、と考えます。

7.4 遷移規則を多項式で書く

現在状態を (X_1, X_2)、次状態を (Y_1, Y_2) と書きましょう。
フィボナッチ遷移なら、

$$Y_1 = X_2, \qquad Y_2 = X_1 + X_2$$

です。

これを「0 になる多項式」として書き換えると、

$$P_1(X, Y) = Y_1 - X_2$$
$$P_2(X, Y) = Y_2 - X_1 - X_2$$

です。
各ステップで

$$P_1 = 0, \qquad P_2 = 0$$

が成り立てば、そのステップの遷移は正しい。

これが AIR の核心です。

AIR の見方

AIR は「正しい計算とはどんな局所法則を満たすか」を書く仕様書です。
言語仕様や回路仕様に相当するものを、多項式制約に翻訳したものだと思うとよいです。

7.5 境界条件（boundary constraints）

遷移規則だけでは、どこから始まった計算か分かりません。
そのため STARK では、**境界条件**も一緒に扱います。

フィボナッチ例なら、

$$a_0 = 1, \qquad b_0 = 1$$

が初期条件です。
さらに公開したい出力条件として、

$$b_4 = 8$$

を入れてもよい。

なぜ必要か

遷移規則だけなら、「どこか別の初期値から始まる別の列」でも満たしてしまうからです。
証明したいのは、**その初期値・その出力**を持つ計算の正しさです。

7.6 AIR を “局所制約” と見る

STARK の非常に大事な感覚はこれです。

> **巨大な計算でも、1 行先との関係は局所的である。**

フィボナッチなら、各時刻の更新はたった二つの式で書けます。
もし 100 万ステップあっても、規則そのものは変わりません。

この「局所規則の繰り返し」が、STARK のスケーラビリティの源です。

CPU 実行の比喩

CPU が長いプログラムを実行するときも、本質的には「1 クロックごとの状態遷移」の繰り返しです。
STARK は、この思想を代数的に押し進めたものと見られます。

7.7 列を多項式として補間する

実行トレースの各列を、そのまま表として持つのではなく、多項式にします。

たとえば時刻集合 $H = \{0, 1, 2, 3, 4\}$ のような領域の上で、
列 $(a_0, a_1, a_2, a_3, a_4)$ を補間して多項式 $A(X)$ にし、
同様に $B(X)$ を作ります。

すると

$$A(t) = a_t, \qquad B(t) = b_t \quad (t \in H)$$

です。

何が嬉しいか

多項式にすることで、

- 低次数性が語れる
- より大きい領域に拡張できる
- 制約を多項式恒等式として書ける

ようになります。

ここで STARK は、「表そのもの」ではなく「表を生成する低次数多項式」に注目し始めます。

7.8 AIR 制約をトレース多項式に持ち込む

フィボナッチ例の AIR 制約

$$Y_1 - X_2 = 0, \qquad Y_2 - X_1 - X_2 = 0$$

を、トレース多項式で表すと、ざっくり

$$A(\mathrm{next}(x)) - B(x) = 0$$
$$B(\mathrm{next}(x)) - A(x) - B(x) = 0$$

のような関係になります。

ここで $\mathrm{next}(x)$ は、「時刻を 1 つ進めた点」に対応します。
実装や文献では、この “1 ステップ先” を有限体上で自然に表すよう、
評価領域を工夫して選びます。

重要な直観

行ごとに「次行との差」をチェックする代わりに、
列多項式の間に成り立つ代数的関係として見るわけです。

7.9 低次数が重要な理由を AIR の文脈で見る

ここで初学者が疑問に思うのは、

> 「遷移規則が正しいだけなら、なぜ低次数まで必要なのか」

という点です。

答えは、**低次数性がないと、証明者が各点で都合よく値を合わせた “偽の表” を作れてしまう**からです。

低次数が与える一貫性

- ある列は短いトレースから来た一つの多項式の評価である
- したがって、点ごとの値はバラバラには選べない
- 少数のランダム検査でも、多項式全体の一貫性が使える

つまり AIR は「何を満たせば正しい計算か」を定め、
低次数性は「その計算表がでたらめな寄せ集めでない」ことを保証します。

7.10 AIR の一般形

一般の AIR は、次のような要素から成ります。

状態幅（width）

1 行あたりに何個の状態変数を持つか。
レジスタ数だと思えばよいです。

遷移制約の数

1 ステップの遷移が、何本の多項式方程式で記述されるか。

遷移制約の次数

各制約多項式の次数。
高すぎると後の証明コストが増えます。

サイクル数（cycle count）

実行ステップ数。
トレースの長さに相当します。

境界条件

初期状態や最終出力など、特定位置で満たすべき条件。

STARK の設計では、これらのパラメータを小さくすることが重要です。

7.11 AIR は回路と何が違うのか

Ligero は回路的な見方が自然でした。
では AIR は回路とどう違うのでしょうか。

回路の見方

- 各ゲートの接続を明示する
- 依存関係は DAG 的
- 一回限りの大きな制約系として見る

AIR の見方

- 状態が時間とともに更新される
- 同じ局所規則が繰り返される
- 長い逐次計算に自然

したがって、**計算を「時間発展するプロセス」と見たいか、「一つの回路網」と見たいか**で、自然な表現が変わります。

7.12 フィボナッチ例を AIR としてまとめる

ここで例をきれいにまとめます。

状態

$$(X_1, X_2) = (a_t, b_t)$$

遷移制約

$$P_1(X, Y) = Y_1 - X_2 = 0$$
$$P_2(X, Y) = Y_2 - X_1 - X_2 = 0$$

初期条件

$$a_0 = 1, \qquad b_0 = 1$$

出力条件

$$b_4 = 8$$

これだけで、「フィボナッチを 4 ステップ回した計算」というオブジェクトが
代数的仕様として書けました。
次章では、この仕様をどうやって**低次数検査の問題**に変えるかを見ます。

7.13 この章のまとめ

- AIR は、計算を**有限体上の状態遷移多項式系**として表したものである。
- 実行トレースは「行が時刻、列がレジスタ」の表である。
- 遷移規則は、現在状態 X と次状態 Y の間の多項式制約 $P_i(X, Y) = 0$ として書ける。
- 初期値や出力値は境界条件として表す。
- STARK では、トレース列を多項式として補間し、その低次数性と制約成立を証明する。
- AIR は、STARK の “何が正しい計算か” を定義する仕様書だと理解するとよい。

練習問題

1. フィボナッチ例で、状態を (a_t, b_t) としたときの遷移制約を自分で書いてください。
2. 境界条件が必要な理由を説明してください。
3. 「遷移規則が成り立つ」ことと「低次数性がある」ことは、どのように役割が違うか説明してください。
4. 自分で簡単な状態遷移（たとえばカウンタ）を一つ考え、AIR 風に書いてみてください。

第 8 章 FRI と ALI：低次数性をどう証明するか

前章で、STARK がまず AIR によって「何が正しい計算か」を多項式制約として書くことを見ました。
しかしそれだけでは足りません。

証明者はまだ、勝手な表を持ち出して

- たまたま見られた場所だけ整合している
- それ以外はでたらめ

ということができてしまいます。

そこで必要なのが、この章の主役である

- **LDE（低次数拡張）**
- **Merkle コミットメント**
- **ALI（または composition による制約の集約）**
- **FRI（低次数近接性検査）**

です。

この章では、STARK の各部品が何を担当しているのかを、役割ごとに分けて理解します。

8.1 まず役割分担を先に言う

STARK の中核部品は、よく混同されます。
最初に役割を一枚の表にしておきます。

部品	役割
AIR	何が正しい計算かを定義する
トレース多項式	実行トレースの各列を表す
LDE	トレース多項式を大きい領域に冗長化する
Merkle 木	大きな評価表にコミットする
ALI / composition	多数の制約を少数の多項式へ集約する
FRI	その多項式が本当に低次数に近いことを検査する
Fiat–Shamir	対話を非対話証明へ変換する

この表を頭に置いて読めば、用語の迷子になりにくいです。

8.2 LDE：なぜ大きい領域で評価し直すのか

トレース多項式 $A(X), B(X), \ldots$ は、もともとは小さな領域 H 上の値を補間して得られます。
しかし STARK では、そこからさらに大きな領域 L で評価し直します。
これが **Low-Degree Extension（LDE）**です。

直観

短い表だけを持っていると、ランダム検査の余地が少ない。
そこで評価点を増やして冗長化し、
「この列は低次数多項式から来ている」という構造を見やすくします。

何が嬉しいか

- 冗長性が増える
- ランダム照会の自由度が増える
- 制約評価や FRI が回しやすくなる
- Merkle 木で大きなオラクルとして扱える

Ligero との似ている点

Ligero でも、短い行を多数点評価して RS 符号語にしました。
STARK の LDE も発想としては近いですが、対象が「行」ではなく「トレース列」や「制約多項式」である点が異なります。

8.3 Merkle 木：大きなオラクルを固定する

LDE すると、証明者が持つ評価表は大きくなります。
検証者がそれを全部受け取るのは現実的ではありません。
そこで証明者は、その評価表を **Merkle 木** でコミットします。

流れ

1. LDE で得た長い評価列を葉に並べる
2. ハッシュで Merkle root を作る
3. root だけを先に公開する
4. 後で検証者が指定した位置だけ、値と認証パスを開く

何を防ぐのか

検証者のランダム質問を見てから、証明者が都合よく表を作り直すことを防ぎます。
つまり、**“最初に表を固定させる”** のが Merkle の役割です。

大事な点

Merkle 木は低次数性を証明しません。
それは FRI の仕事です。
Merkle 木はあくまで**コミットメント**です。

8.4 ALI：多数の制約をどうやってまとめるか

AIR には、多数の遷移制約や境界条件があります。
これを一本ずつ別々に検査すると重すぎます。
そこで STARK では、それらをランダム係数でまとめます。
これが **ALI（Algebraic Linking IOP）** の直観です。

何をしているのか

- 遷移制約 $P_1, P_2, \dots, P_s$ がある
- 各制約が各時刻で 0 になるべき
- それらをランダム係数 $\alpha_1, \dots, \alpha_s$ で混ぜて

$$\alpha_1 P_1 + \cdots + \alpha_s P_s$$

のような一つの合成制約にする

なぜ効くのか

どれか一つでも制約が壊れていれば、ランダムに混ぜた結果も高確率で壊れます。
これは Ligero のランダム線形結合と極めて近い思想です。

“linking” の意味

各列多項式や境界条件が、正しい関係に “つながっている” ことを、一つの代数的対象に束ねる。
この意味で ALI は「制約の糊付け」の役割を持ちます。

8.5 composition polynomial という見方

現代の STARK 解説では、ALI という語よりも
composition polynomial という言い方が前面に出ることがあります。

直観

たくさんの制約違反を、一つの多項式 $C(X)$ の非零性に押し込める、という見方です。
正しい計算なら $C(X)$ は低次数で、しかも特定領域で適切な振る舞いをします。
不正があれば、その性質が壊れます。

初学者にとっての理解

ALI と composition polynomial は、詳細には文献流儀の違いもありますが、
本書ではまず

> **“たくさんの制約をランダムに混ぜて、少数の多項式検査へ圧縮する部分”**

として捉えれば十分です。

8.6 低次数性を本当にどう調べるか：FRI

ここでようやく **FRI** が登場します。
FRI は **Fast Reed–Solomon Interactive Oracle Proof of Proximity** の略で、

> **この長い評価表が、本当に低次数多項式の評価列に近いか**

を検査するプロトコルです。

“近い” とは

評価表が、ある RS 符号語 —— つまりある低次数多項式の評価列 —— と
ハミング距離の意味で十分近いかどうかを見ます。

なぜ “proximity” なのか

完全に一致しているかを全部読むのは不可能です。
そこで、「低次数コードから十分遠いなら高確率で拒否する」という近接性検査を行います。

8.7 FRI の直観：大きな問題を半分にたたむ

FRI の最も大事な感覚は、

> **低次数らしさの検査を、再帰的に “半分サイズの問題” へたたむ**

ことです。

ごく粗い絵

長さ 16 の評価列があるとします。
FRI はこれを、一回のラウンドで長さ 8 の新しい列へと “折りたたみ” ます。
さらにそれを長さ 4、長さ 2、最後は定数へとたたんでいきます。

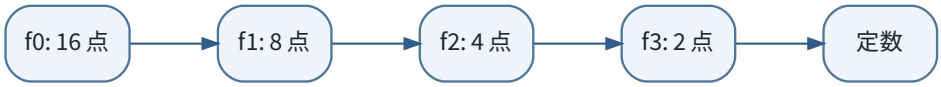

図 5: FRI の折りたたみイメージ

各段階で、前の列と次の列が整合しているかを少数クエリで検査します。

なぜこれが低次数と関係するのか

もし元の列が本当に低次数多項式の評価列なら、
適切に折りたたんだ次の列もまた低次数多項式の評価列であるはずです。
逆に、元の列が低次数から遠ければ、この整合性がどこかで壊れます。

8.8 FRI をもう少しだけ具体的に

FRI の詳細はかなり技術的ですが、直観だけを追うとこうです。

1. 証明者は、元の評価表 $f^{(0)}$ に対して、半分サイズの新しい表 $f^{(1)}$ を用意する
2. 検証者はランダム点を選び、
   - $f^{(0)}$ の少数点
   - $f^{(1)}$ の対応点 を問い合わせる
3. それらが「折りたたみ規則」に従うかを確かめる
4. 同じことを再帰的に繰り返す
5. 最後に、定数や非常に低次数のものまで落ちたら受理する

どこが “Fast” なのか

FRI が重要なのは、原論文系の文脈で

- 証明者計算が線形時間
- 検証者計算が対数的

という非常に良いスケーリングを与えたからです。

これが STARK 全体の実用性に大きく貢献しました。

8.9 FRI は何を証明し、何を証明しないか

ここを混同しないことが大切です。

FRI が証明するもの

- ある長い評価表が、低次数多項式の評価列に近いこと

FRI が直接は証明しないもの

- その多項式が正しい計算から来たこと

- 境界条件を満たすこと
- AIR 制約を満たすこと

それらは、ALI / composition の段階で「低次数であるべき対象」に埋め込まれています。
つまり

- **AIR / ALI が “何を証明すべきか” を作り**
- **FRI が “それが本当に低次数か” を検査する**

という役割分担です。

8.10 零知識は STARK でどう入るか

ここまでだと、トレース多項式の値をそのまま見せることになり、秘密が漏れかねません。
そのため STARK では、公開するオラクルに対して**ブラインディング**や**マスキング**を入れます。

直観

- トレース多項式そのものを裸で出さない
- ランダムな低次数成分を足す
- ただし、制約検査が壊れないように慎重に設計する

注意

零知識化の具体手法は、STARK の世代や実装によって細部がかなり異なります。
本書では、まず

> **“トレースをそのまま見せないために、制約を保ったままランダム化する必要がある”**

という理解で十分です。

8.11 Fiat–Shamir と非対話化

対話型の FRI/ALI は、そのままでは検証者からの乱数送信が必要です。
非対話型 STARK にするには、Ligero と同様、

- 各 Merkle root
- それまでの transcript
- 必要なら公開入力

をハッシュして、次のチャレンジを導きます。

こうして、

1. 証明者が root を出す
2. その root からハッシュで乱数を導く
3. 次のオラクルを出す
4. またハッシュで乱数を導く

という流れで、検証者が実際にはいなくても transcript を構成できます。

8.12 STARK の各部品を Ligero と見比べる

ここまで来ると、Ligero との共通性も見えてきます。

観点	Ligero	STARK
ランダム線形結合	行や制約をまとめる	AIR 制約を composition にまとめる
符号化	各行を RS 符号化	トレース列や composition を低次数コードとして扱う
コミットメント	Merkle で表にコミット	Merkle でオラクルにコミット
spot-check	ランダム列を開く	ランダム位置を開く
低次数検査	各種整合性検査の中に現れる	FRI が中核として担う

つまり両者は、違う世界観を持ちながらも、

- ハッシュ
- 符号
- ランダム化
- 局所照会

を共有しているのです。

8.13 この章のまとめ

- LDE は、短いトレースを大きい領域へ拡張して冗長化する。
- Merkle 木は、大きな評価表を先に固定させるためのコミットメントである。
- ALI / composition は、多数の AIR 制約をランダムにまとめて少数の多項式検査へ圧縮する。
- FRI は、その多項式が本当に低次数コードに近いかを再帰的に検査する。
- STARK の零知識化には、オラクルのブラインディングが必要である。
- 役割分担を一言で言えば、**AIR が仕様、ALI が集約、FRI が低次数検査、Merkle が固定**である。

練習問題

1. LDE が必要な理由を、「評価点を増やす意味」という観点から説明してください。
2. Merkle 木と FRI の役割の違いを整理してください。
3. ALI / composition が、Ligero のランダム線形結合とどのように似ているか説明してください。
4. FRI の「半分にたたむ」という直観を、自分の言葉で言い換えてください。
5. FRI が “低次数性” だけを扱い、“計算の正しさそのもの” は別の部品が担う、という役割分担を説明してください。

第 9 章 zk-STARK をフィボナッチで追う

この章では、zk-STARK を**フィボナッチ列**の例で最後まで追います。
もちろん本物の STARK 実装はもっと大きく、もっと複雑です。
しかし「何にコミットし、何を低次数として示し、何をランダムに開くのか」は、小例でも十分に見えます。

9.1 証明したい主張

公開主張を次のようにしましょう。

> **初期値 $(1, 1)$ から始めてフィボナッチ遷移を 4 回行うと、最終的に $b_4 = 8$ になる。**

この主張自体は手で確かめられます。
しかし STARK の目的は、この小さな例を通じて**大きな計算でも同じ骨格が働く**ことを見ることです。

9.2 実行トレースを作る

状態は 2 レジスタ

$$(a_t, b_t)$$

とし、遷移規則は

$$(a_{t+1}, b_{t+1}) = (b_t, a_t + b_t)$$

です。
するとトレースはこうなります。

t	a_t	b_t
0	1	1
1	1	2
2	2	3
3	3	5
4	5	8

この表が、STARK における witness の中心です。
ブロックチェーンなら、これが巨大な VM 実行トレースになります。

9.3 AIR を書く

前章の内容をそのまま使うと、AIR は次の通りです。

遷移制約

$$P_1(X, Y) = Y_1 - X_2 = 0$$
$$P_2(X, Y) = Y_2 - X_1 - X_2 = 0$$

境界条件

$$a_0 = 1, \qquad b_0 = 1, \qquad b_4 = 8$$

これで、「正しいフィボナッチ計算」とは何かが決まりました。

9.4 列を多項式にする

トレースの 2 本の列

$$(1, 1, 2, 3, 5), \qquad (1, 2, 3, 5, 8)$$

を、それぞれ多項式 $A(X), B(X)$ に補間します。

初学者向けの理解

ここで本当に必要なのは、「列がたくさんの数の表ではなく、低次数多項式として扱われる」という視点です。
多項式そのものを明示的に書かなくても、
「A と B は時刻点でこの値を取る低次数多項式」と理解すれば十分です。

9.5 小領域から大領域へ：LDE

補間しただけでは、まだトレースは短いです。
そこで A と B を、より大きな領域 L 上で評価します。

たとえば概念的には

- 元の時刻点集合 H は 5 点
- 拡張領域 L はもっと多い点を持つ

とします。

すると、

- A の LDE
- B の LDE

という二つの長い評価表が得られます。

何が起きたか

元の短いトレース列が、長いオラクルへ変わりました。
この長いオラクルが Merkle 木でコミットされ、ランダム照会の対象になります。

9.6 Merkle でコミットする

証明者は LDE された評価表全体を Merkle 木に入れ、root を公開します。
これで、

- トレース列はもう後から変更できない
- しかし検証者は全部を読む必要がない

という状態になります。

この時点で検証者が知っていること

- まだ中身は知らない
- ただし「ある大きな表」が root によって固定されたことは知っている

STARK の証明は、この “固定されたが見えていない表” に対して進みます。

9.7 制約を一つにまとめる

AIR 制約は本来たくさんの点でチェックされるべきです。
しかしそれを全部やるのは重い。
そこでランダム係数を使って、制約を一つの composition polynomial にまとめます。

フィボナッチ例の直観

制約は主に三種類あります。

1. 遷移制約 $Y_1 - X_2 = 0$
2. 遷移制約 $Y_2 - X_1 - X_2 = 0$
3. 境界条件 $a_0 = 1, b_0 = 1, b_4 = 8$

これらにランダム係数 $\alpha, \beta, \gamma, \ldots$ を掛けて足し合わせ、
「正しいトレースなら、この合成多項式は低次数になるはずだ」という形へ変換します。

重要な点

検証者は、個々の制約がどこで壊れているかを逐一見ません。
ランダムに混ぜた結果だけを見ます。
この思想は Ligero と非常によく似ています。

9.8 もし最終値を偽装したらどうなるか

仮に証明者が、最後の値を 8 ではなく 9 にした偽トレースを持ち出したとします。

t	a_t	b_t
0	1	1
1	1	2
2	2	3
3	3	5
4	5	9

このとき、少なくとも

- 遷移制約が壊れる
  （$5 + 3 = 8$ なのに 9 になっている）
- あるいは境界条件 $b_4 = 8$ が壊れる

のどちらかです。

その結果、composition polynomial の “低次数であるはず” という性質が崩れます。
FRI はそこを突きます。

9.9 FRI をこの例で見る

FRI は、「その合成オラクルが低次数に近いか」を検査します。

ごく粗い流れ

1. 証明者は、composition polynomial の評価表を用意する
2. それも Merkle でコミットする
3. 検証者はランダムチャレンジに応じて、折りたたみ系列

$$f^{(0)}, f^{(1)}, f^{(2)}, \ldots$$

   を検査する
4. 各段階で少数点だけ開く
5. 最後に、定数近くまでたためるなら受理する

初学者向けの見方

FRI は「**この大きな表は、本当に一本の低次数多項式から来たのか？**」を、
全部読まずに確認するための再帰的検査です。

9.10 検証者は具体的に何を見るのか

STARK の検証者は、直観的には次のものを見ます。

1. いくつかの Merkle root
   これで大きな表が固定される
2. ハッシュから導いたランダムチャレンジ
   どの制約をどう混ぜるか、どこを開かせるかを決める
3. ランダム位置の値と Merkle path
   本当に root に整合する値かどうかを見る
4. FRI の折りたたみ整合性
   低次数らしさが段階ごとに保たれているかを見る

ここでのポイントは、**計算全体の全表は見ていない**ことです。
見ているのは、ごく少数の値と、それらの間の代数的関係だけです。

9.11 なぜ少数点で十分なのか

この例を通じて分かるのは、STARK の検査力が三層構造になっていることです。

第 1 層：コミットメント

証明者は表を後出し変更できない。

第 2 層：ランダム集約

多数の制約を、ランダムに混ぜた少数の制約へ圧縮する。

第 3 層：低次数検査

圧縮後の対象が本当に低次数多項式由来かを FRI で見る。

この三層がそろうことで、「少数点だけ見れば十分」が成立します。

9.12 零知識はこの例でどう効くか

フィボナッチ例はそもそも秘密が少ないので、零知識のありがたみが見えにくいです。
しかし実際の応用では、トレースは秘密情報を大量に含みます。

たとえば

- 秘密鍵計算
- 取引の中間状態
- 非公開データベースへの照会
- 機密な業務計算

などです。

そのため STARK では、トレース多項式や制約オラクルにブラインディングを入れ、
開かれる少数点からでも内部状態が再構成されにくいようにします。

9.13 フィボナッチから実運用への橋渡し

「フィボナッチでは簡単すぎる」と感じるかもしれません。
その通りです。
しかし実運用では、同じ構図がそのまま大きくなります。

フィボナッチ例

- レジスタ数が少ない
- 遷移規則が単純
- ステップ数が少ない

ロールアップや VM 実行

- レジスタ・メモリ・命令ポインタなど多数の状態
- 命令ごとに複雑な AIR
- ステップ数が膨大
- 入出力やメモリ整合性も境界条件に入る

違うのは規模であって、骨格は同じです。

9.14 この章のまとめ

- zk-STARK の小例では、**実行トレース → AIR → 列の補間 → LDE → Merkle → composition → FRI** という流れが確認できる。
- 検証者はトレース全体を見ず、ランダム位置の少数開示だけを見る。
- 制約違反があると composition polynomial の低次数性が壊れ、FRI に捕まりやすくなる。
- フィボナッチ例は小さいが、巨大な VM 実行やロールアップも同じ骨格で理解できる。

練習問題

1. フィボナッチ例の実行トレースを、自分で 5 ステップ分書いてみてください。
2. 最終値を 9 に偽装したとき、どの制約が壊れるか説明してください。
3. Merkle root、composition polynomial、FRI がそれぞれ何を担当しているか整理してください。
4. フィボナッチではなく、簡単なカウンタ更新規則に対して STARK の流れを口頭で説明してみてください。

第 10 章 Ligero と zk-STARK を比較する

ここまでで、Ligero と zk-STARK の骨格を別々に見てきました。
この章では、それらを**同じ土俵で比較**します。

比較の目的は、「どちらが上か」を決めることではありません。
重要なのは、

> **どんな計算に、どんな設計思想が自然か**

を見抜けるようになることです。

10.1 まず一言で違いを言う

- **Ligero** は、**回路制約の満足性**を、符号化された行列のランダム検査に落とす。
- **zk-STARK** は、**実行トレースの正しさ**を、低次数多項式の近接性検査に落とす。

この一文が、ほぼすべての違いの出発点です。

10.2 比較表

最初に全体表を置きます。
細部はこのあと順に解説します。

観点	Ligero	zk-STARK
主な見方	回路・行列・符号	実行トレース・AIR・FRI
証明対象の自然な表現	算術回路満足性	長い逐次計算・状態遷移
コアとなる代数構造	interleaved RS 符号	低次数トレース多項式と RS 近接性
多数制約の圧縮	ランダム線形結合	ALI / composition polynomial
コミットメント	ハッシュ / Merkle	Merkle
trusted setup	不要	不要
量子耐性の立場	ハッシュ・符号中心で比較的親和的	同様にハッシュ・符号中心で比較的親和的
典型的な強み	中規模〜大規模回路、単純な部品	非常に大きな逐次計算、ロールアップ
典型的な弱み	超巨大トレース系では相対的に不利	証明サイズや実装の重さ
証明の印象	“表を spot-check する”	“トレース全体を多項式化する”

10.3 何を圧縮しているのか

この観点が最も重要です。

Ligero が圧縮するもの

Ligero は、回路中の大量の配線値・制約を**行列**に並べ、
その行や制約をランダム線形結合で圧縮します。

つまり、

> **“回路の静的な制約系” を圧縮する**

という色が強いです。

STARK が圧縮するもの

STARK は、計算過程の全履歴を**実行トレース**として持ち、
その列を低次数多項式として表します。

つまり、

> **“時間発展する計算の軌跡” を圧縮する**

という色が強いです。

この差は本質的です。
Ligero では「制約の束をどう spot-check するか」が中心であり、
STARK では「長い軌跡をどう低次数として記述するか」が中心です。

10.4 回路 vs トレース

回路が自然な問題

- ハッシュ前像の知識
- 固定サイズの認証回路
- 中規模の算術チェック
- “この relation を満たす witness がある” という静的な問題

このような問題では、Ligero 的な見方が自然です。

トレースが自然な問題

- 仮想マシン実行
- 状態更新の長い列
- 多数ステップのアルゴリズム実行
- ロールアップや CPU/VM の遷移

このような問題では、STARK 的な見方が自然です。

重要な補足

もちろん、回路をトレースに変換することも、トレースを回路化することも原理的には可能です。
大事なのは、**どちらの表現が無理なく小さなパラメータで書けるか**です。

10.5 ランダム線形結合という共通骨格

Ligero と STARK はかなり違って見えますが、深いところで共通点もあります。

Ligero での役割

- 行をランダムに混ぜる
- 線形制約をランダムに混ぜる
- 乗算制約もランダムに混ぜる

STARK での役割

- AIR の多数の制約をランダム係数で composition にまとめる
- 場合によっては複数のオラクルをまとめて一つの低次数検査へつなぐ

本質

どちらも

> **“たくさんある条件を、一つのランダムな条件に圧縮する”**

という戦略を取っています。

初学者にとってこれは重要な発見です。
「Ligero と STARK はまったく別世界」ではあるが、
制約圧縮の哲学は共通しているのです。

10.6 低次数性の使い方の違い

Ligero

低次数性は主として

- 各行が RS 符号語であること
- 制約チェックに現れる多項式が整合すること

に使われます。
低次数性は重要ですが、FRI のようにそれ自体が巨大な独立コンポーネントとして前面に出る印象は比較的弱いです。

STARK

低次数性は完全に主役です。

- トレース列が低次数多項式から来る
- composition polynomial が低次数である
- FRI がその近接性を検査する

というように、**低次数性の検査そのもの**がプロトコルの中心になります。

10.7 証明サイズ・検証時間・証明時間の感覚

ここは一律に比較しにくいので、**定性的な感覚**として整理します。

Ligero

- 通信量は回路サイズに対してサブリニア
- 中規模回路でも具体的な意味を持つよう設計されている
- 複数インスタンスで償却が効きやすい

STARK

- 巨大な逐次計算でも検証が比較的軽い
- transparent で公開検証しやすい
- ただし証明サイズは小さくはないことが多い
- 証明者はかなり重い

比較時の注意

「どちらが速いか」は、次に大きく依存します。

- 問題の表現（回路 / トレース）

- 実装成熟度
- 体の選択
- セキュリティ水準
- バッチの有無
- ハードウェア最適化

したがって、一般論として断定しすぎてはいけません。

10.8 透明性の意味では両者は近い

Ligero も zk-STARK も、trusted setup を要しない方向にある点で共通しています。
これは pairing-based SNARK との大きな違いです。

ただし、**透明性だけを理由に両者を同一視してはいけません**。
透明性は共通でも、内部の代数化は大きく異なります。

10.9 零知識の入れ方の違い

Ligero

- マスキングや MPC 的なプライバシーの視点が重要
- 一部のビューや列を開いても witness を漏らさないよう設計する

STARK

- トレース多項式や制約オラクルにブラインディングを加える
- 開かれる少数点から内部状態が漏れにくいようにする

共通点

どちらも、単に「全部見せない」だけでは零知識になりません。
公開される部分自体が、ランダム化されていても検証が崩れないことが必要です。

10.10 実装のしやすさという意味での違い

Ligero の実装感

- RS 符号化
- 行列配置
- ランダム線形結合
- ハッシュコミットメント

という比較的見通しのよい部品で組み上がる感覚があります。

STARK の実装感

- AIR をどう設計するか
- LDE をどう高速に行うか
- Merkle の実装
- FRI の再帰
- zero-knowledge masking
- 実際のフィールドと FFT 的処理

など、多層の実装が必要です。

ただし

「概念の階層が多い = 実装が難しい」と単純には言えません。
一度エコシステムが整うと、STARK 側は巨大計算に対して非常に強力な再利用性を持ちます。

10.11 どんなときに Ligero を考えるか

Ligero を考えたくなるのは、たとえば次のようなときです。

- 回路として表しやすい relation を証明したい
- trusted setup を避けたい
- 証明系の構造を比較的素直に理解・研究したい
- 同種の多数インスタンスをまとめたい
- 極端に大きい VM 実行より、回路満足性の方が自然な問題である

10.12 どんなときに STARK を考えるか

STARK を考えたくなるのは、たとえば次のようなときです。

- 長い逐次計算を証明したい
- ロールアップや VM 実行のように、状態遷移が本質の問題である
- 透明性を重視する
- 公開検証やオンチェーン検証の文脈が強い
- トレース表現・AIR 表現が自然に書ける

10.13 “どちらを選ぶか” の雑な指針

かなり荒いが実践的な指針を挙げると次のようになります。

Ligero 寄り

- 問題がもともと回路制約として自然
- 中規模〜大規模回路
- 証明系の部品を見通しよく把握したい
- batching / amortization を重視

STARK 寄り

- 問題が長い状態遷移として自然
- 実行長が非常に大きい
- 透明な公開検証基盤を作りたい
- ロールアップや VM の枠組みに乗せたい

別系統も考えるべき場合

- きわめて短い証明サイズが最優先なら pairing-based SNARK 系
- relation が小さく特殊なら専用プロトコル
- 2 者対話で十分ならシグマプロトコル系

このように、Ligero vs STARK は「ZK の全世界の二択」ではありません。

10.14 この章のまとめ

- Ligero と STARK の最大の違いは、**回路を圧縮するか、実行トレースを圧縮するか** にある。

- Ligero は **行列・符号・spot-check** の世界、STARK は **トレース・AIR・FRI** の世界で理解するとよい。
- ただし両者とも、ランダム線形結合・ハッシュコミットメント・低次数構造を共有している。
- 使い分けは、問題の自然な表現と、求めるスケーリング特性によって決まる。

練習問題

1. Ligero と STARK の違いを「何を圧縮しているか」の観点から 3 行で説明してください。
2. 低次数性が Ligero と STARK でどのように違う位置づけを持つか説明してください。
3. ある計算問題が「回路向き」か「トレース向き」かを判定する基準を考えてください。
4. ロールアップに STARK が自然な理由を、状態遷移の観点から説明してください。

第 11 章 ユースケース：どこで役立つのか

この章では、Ligero と zk-STARK がどのような場面で役立つのかを整理します。
ここで大事なのは、「ゼロ知識証明は秘密を隠す道具」という一面だけでなく、

> **巨大な計算の正しさを、誰でも軽く検証できるようにする道具**

でもある、という点です。

11.1 ユースケースを三種類に分ける

ゼロ知識証明の用途は大きく三つに分けると見通しがよくなります。

1. 秘密を隠したまま条件成立を示す

- パスワードや鍵の知識
- ハッシュ前像の知識
- 認証情報の保有
- 非公開データに対する制約充足

2. 計算の正しさを軽く検証する

- 外部委託計算
- クラウド上の集計
- 長いアルゴリズムの実行
- 機密データ上の分析

3. 公共的・分散的な検証を可能にする

- ブロックチェーンの状態更新
- ロールアップ
- 監査可能な自治システム
- 公的データに関する検証可能な主張

Ligero も STARK もこの三類型にまたがりますが、
得意分野は少し異なります。

11.2 パスワード・ハッシュ・認証

最もわかりやすいのは、「秘密値を見せずに、ある条件を満たすことだけを示す」型です。

例：ハッシュ前像の知識

公開値 h に対して、

$$H(w) = h$$

を満たす w を知っていることを証明する。
SHA-256 のようなハッシュ回路は、まさに Ligero の原論文でも代表例として扱われました。

どちらが向くか

- relation が固定サイズの回路として自然なら **Ligero 的** な見方がしやすい
- 同種の多数証明をまとめたいなら、Ligero の償却性が魅力になることがある

ただし、現実の認証プロトコルとして安全に使うには、
再利用攻撃やプロトコル束縛など別の設計問題もあります。

11.3 秘匿監査・外部委託計算

企業や機関が機密データを持っていて、その上で行った計算の正しさだけを示したい、
という状況は極めて一般的です。

例

- 売上台帳を公開せずに、税額計算が規則通りであることを示す
- 給与データを公開せずに、集計結果が正しいことを示す
- 顧客データを公開せずに、抽出条件を満たす件数だけを示す

Ligero の見方

固定された計算回路を証明したいとき、
特に relation があらかじめ明確なら Ligero 的な回路化は自然です。

STARK の見方

長いログ処理や状態更新列のように、
計算が逐次的で大きいなら STARK 的なトレース化が自然です。

11.4 大規模状態更新とブロックチェーン

STARK が現実に最も強く存在感を持つのはここでしょう。

何をしたいのか

多数のトランザクションをオフチェーンで処理し、
その結果得られた新しい状態が正しいことを、オンチェーンでは**一つの証明**で示したい。

なぜ STARK と相性がよいか

ブロックチェーンの状態更新は、本質的に

- 前状態
- 取引列
- 遷移規則
- 新状態

という**長い状態遷移**です。
したがって、実行トレースと AIR の発想が非常に自然に当てはまります。

典型的な流れ

1. オフチェーンで大量の取引を実行する
2. 実行トレースを作る
3. AIR で正しい遷移を定義する
4. STARK 証明を作る
5. オンチェーンではその証明だけを検証する

この構図が、validity rollup の中心です。

11.5 StarkEx と Starknet は何を示しているか

現在の STARK の実運用例として、**StarkEx** と **Starknet** がよく挙げられます。
これらは、STARK が単なる理論にとどまらず、

- 大量取引のバッチ検証
- オンチェーン検証
- スケーリング基盤

として実働していることを示す代表例です。

教科書的に重要な点

ここで本当に重要なのは具体的な数値そのものより、

> **STARK は「長い状態遷移の正しさ」を一つの証明へ圧縮する技術として、実運用とよく噛み合う**

という事実です。

11.6 DNA データベース例が示すもの

STARK 原論文の DNA プロファイル照合の例は、
ブロックチェーンとは別方向の重要性を示しています。

この例の本質

- データは機密
- しかし公共的検証可能性が必要
- 計算対象は大きい
- 出したいのは yes/no や条件成立だけ

これは現代社会で非常に一般的な要求です。

類似する応用

- 医療データ上の条件チェック
- 非公開検索の検証
- 行政・監査における整合性証明
- 機密データベースに対するクエリ結果の正しさ証明

STARK はこうした「**秘匿と公共的検証の両立**」に強い魅力を持ちます。

11.7 Ligero の使いどころを丁寧に捉える

STARK の実運用事例は目立ちますが、だからといって Ligero が価値を失うわけではありません。

Ligero が有効な場面

- relation が回路として自然に書ける
- 比較的単純な暗号部品で構成したい
- trusted setup を避けたい
- 多数の同種インスタンスをまとめたい
- 証明系の内部を教育・研究目的で見通しよく扱いたい

研究的な位置づけ

Ligero 系は、BooLigero や Ligero++ のような派生研究も含め、
「透明寄り・軽量志向・回路ベース」の系譜として今でも重要です。

11.8 多数インスタンスの一括証明

Ligero の原論文で強調されたのが、**multi-instance amortization** です。
同じ回路に対する多数の証明要求があるとき、
共有できる部分をまとめることで、一件あたり通信を下げられます。

例

- 多数のユーザが同じ検査回路を通る
- 同じハッシュ関数に対する多数の証明
- バッチ化された証明要求

このような場面では、Ligero 的発想が非常に自然です。

11.9 ゼロ知識証明は “秘密保持” だけの技術ではない

初学者はしばしば、ゼロ知識証明を「プライバシー保護の技術」とだけ理解します。
もちろんそれは正しい一面ですが、半分しか見ていません。

もう半分は、

> **“計算の完全再実行なしに、正しさを納得させる” 計算検証技術**

です。

Ligero でも STARK でも同じ

- 証明者は大量の内部計算をした
- 検証者はそのごく一部だけを見る
- それでも高確率で正しさが言える

この構図自体が、現代の検証可能計算の核心です。

11.10 どんなときに使わない方がよいか

ユースケースだけでなく、「使わない方がよい場合」も知るべきです。

1. relation が小さく単純すぎる

単純な知識証明なら、シグマプロトコルや専用プロトコルの方がはるかに簡単なことがあります。

2. 証明サイズが極端に重要

オンチェーン calldata や帯域制約が極端に厳しい場合、
STARK や Ligero より、別の succinct 系が向くことがあります。

3. 開発コストが見合わない

零知識証明は強力ですが、関係の定式化、セキュリティレビュー、実装検証が重い。
単純なアクセス制御問題に常に使うべきとは限りません。

11.11 実務的な選び方

かなり粗いが、実務寄りの判断基準を挙げます。

Ligero を考える

- 回路で書きやすい
- trusted setup なしが欲しい
- 同種インスタンスのバッチが多い
- 証明系の構造を簡潔に保ちたい

STARK を考える

- 実行トレースが本質
- 状態遷移が長い
- 公開検証やロールアップ基盤を作りたい
- 透明性を重視する

別系統も検討

- 極小証明サイズ
- 特殊用途専用の relation
- 開発期間や監査コストの制約が厳しい

11.12 この章のまとめ

- Ligero と STARK は、どちらも秘密保持だけでなく**計算の正しさの軽量検証**に使える。
- Ligero は回路ベースの relation や多数インスタンスの償却に向く。
- STARK は長い状態遷移、ロールアップ、公開検証基盤に特に向く。
- ゼロ知識証明は万能ではなく、問題サイズ・証明サイズ・開発コストも含めて選ぶべきである。

練習問題

1. 「ゼロ知識証明は秘密保持だけの技術ではない」と言える理由を説明してください。
2. ブロックチェーンの状態更新が、なぜ STARK と相性がよいのか述べてください。
3. Ligero の multi-instance amortization が役立ちそうな場面を 2 つ挙げてください。
4. 自分の専門分野に引きつけて、Ligero 向き・STARK 向きの問題を一つずつ考えてください。

参考メモ

- STARK 系の実運用例としては、StarkWare の **StarkEx**、**Starknet** が代表的です。
- より新しい運用状況や数値は変動するため、実際に引用するときは公式サイト・公式ドキュメントで最新状況を確認してください。

第 12 章 学習ロードマップと参考文献

本書の目的は、Ligero と zk-STARK の**骨格**を掴み、原論文へ入るための地図を与えることでした。
最後に、ここから先の学び方を整理します。

12.1 どこまで理解できていれば十分か

まず、原論文に進む前に最低限ほしい理解を挙げます。

Ligero について

- なぜ行列化するのか
- interleaved RS 符号とは何か
- ランダム線形結合がなぜ制約圧縮になるのか
- 行方向圧縮と列方向 spot-check が平方根オーダにつながること

STARK について

- 実行トレースとは何か
- AIR が何を定義するか
- ALI / composition が何をしているか
- FRI が何を検査するか
- Merkle 木がなぜ必要か

ここまでが腑に落ちていれば、原論文の記号の洪水にも耐えやすくなります。

12.2 おすすめの読み順

ルート A：初学者向け最短コース

1. 本書を通読する
2. Ligero 原論文の abstract, introduction, overview を読む
3. STARK 原論文の introduction, methods overview を読む
4. FRI の abstract と序論を読む
5. 分からない語を用語集で引き直す

ルート B：理論好き向け

1. PCP / IOP の基本概念を復習
2. Reed–Solomon 符号と低次数検査を確認
3. Ligero 原論文の soundness 周りを読む
4. FRI の再帰と proximity 検査を読む
5. STARK 原論文の AIR / ALI / FRI の接続を見る

ルート C：実装や応用に関心が強い人向け

1. STARK の全体像を再読
2. Starknet / StarkEx の公式資料で “何を証明しているのか” を見る
3. AIR の具体例を自作する
4. 小さな有限体で FRI の折りたたみを手で追う
5. その後に実装資料へ進む

12.3 原論文を読むときのコツ

Ligero の原論文

Ligero の原論文は、最初から細部に潜るとかなり重いです。
次の順に読むとよいです。

1. **Abstract / Introduction**
   何を達成したい論文かを掴む
2. **High-level construction**
   平方根の直観を確認する
3. **Interleaved code testing**
   行の membership test を丁寧に追う
4. **Linear / Quadratic constraint testing**
   回路制約がどう落ちるかを見る
5. **Implementation / benchmarks**
   どの程度の具体性能を狙っていたかを確認する

最初は完全な証明を追い切れなくても構いません。
むしろ「どこが低次数検査で、どこがランダム線形結合か」をマーキングしながら読むと見失いにくいです。

STARK の原論文

STARK 原論文は、技術的にも記述量的にもかなり大きいです。
次の順が勧めやすいです。

1. **Introduction**
   透明性・スケーラビリティ・DNA 例の意味を読む
2. **Methods overview**
   AIR / ALI / FRI の役割分担だけ掴む
3. **Definitions around STIK / STARK**
   透明性やスケーラビリティの定義を確認
4. **AIR section**
   トレースから多項式制約への翻訳を見る
5. **FRI section**
   低次数近接性検査の主役を読む
6. **Appendices**
   必要になった箇所だけ参照する

STARK 論文は Appendix が巨大です。
最初から全部読む必要はありません。

12.4 参考文献（最重要）

以下に、本書の理解に特に重要な文献を挙げます。
細かな書誌情報は版によって少し異なることがありますが、検索しやすいよう題名を優先して載せます。

A. Ligero の中核文献

1. **Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam**
   Ligero: Lightweight Sublinear Arguments Without a Trusted Setup
   CCS 2017.

Ligero の原点。平方根オーダの通信、interleaved RS 符号、線形・二次制約検査、SHA-256 ベンチマークがまとまっています。

2. **Ligero 系の拡張・改良論文（例：Ligero++, BooLigero）**
   Boolean 回路向けの改良や証明サイズ改善を学ぶために有用です。
   原論文の理解後に読むと、「何がボトルネックで、どこが改良点なのか」が見えます。

B. STARK / FRI の中核文献

3. **Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev**
   Scalable, transparent, and post-quantum secure computational integrity
   2018. zk-STARK の原論文。DNA データベース例、透明性、AIR / ALI / FRI の接続が読めます。
4. **Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev**
   Fast Reed-Solomon Interactive Oracle Proofs of Proximity
   FRI の原論文。
   STARK の心臓部である低次数近接性検査を理解するための必読文献です。

C. 周辺理解に有用な文献

5. **PCP / IOP 系の解説文献**
   STARK や Ligero の背景として、PCP、IOP、IOPP の文献を読むと全体像が深まります。
6. **Merkle tree と Fiat–Shamir の基礎文献・解説**
   非対話化や公開検証の感覚を固めるのに有用です。

12.5 公式資料・公式ドキュメント

研究論文とは別に、実運用の文脈を見るには公式ドキュメントが有益です。

Zenn Book の構成確認

本書を実際に Zenn に載せる際には、Zenn の Book 構成・`config.yaml` ・章ファイル規約を公式ドキュメントで確認してください。

Starknet / StarkEx

STARK の実運用を知るには、Starknet や StarkEx の公式資料が参考になります。
ただし数値や運用状況は変わるため、引用のたびに最新確認が必要です。

12.6 自習課題のすすめ

本書を読んで終わりにしないために、次のような小課題を勧めます。

課題 1：自分で RS 符号を作る

$\mathbb{F}_{17}$ 上で一次または二次多項式を選び、
評価点列を作って RS 符号語を書いてみてください。

課題 2：Ligero の membership test を手計算する

2〜3 行の小さな行列を用意し、ランダム係数で線形結合して、
列を 1〜2 個だけ開いて整合性を確認してみてください。

課題 3：自分で AIR を書く

フィボナッチ以外に、カウンタ、線形合同法、簡単な有限状態機械などに対して AIR を書いてみてください。

課題 4：FRI の折りたたみを概念的に追う

長さ 8 の評価表が長さ 4、長さ 2 へと縮む図を自分で描き、
「なぜ低次数なら整合するのか」を言葉で説明してみてください。

課題 5：Ligero と STARK の制約圧縮の類似点を書く

両者ともランダム線形結合を使うことに注目し、
「何を混ぜて、何を得ているのか」を比較表にしてみてください。

12.7 よくある学習上のつまずき

つまずき 1：有限体が直観に合わない

最初は $\mathbb{F}_{17}$ のような小さな素体だけで十分です。
いきなり $\mathbb{F}_{2^{64}}$ を完全理解しようとしなくてよいです。

つまずき 2：FRI が難しすぎる

FRI は多くの読者が最初に苦しむ場所です。
「低次数性を再帰的に半分サイズへ落とす検査」とだけ掴み、
細かい補題は後回しで構いません。

つまずき 3：STARK の記号が多すぎる

AIR、LDE、composition、FRI、Merkle の役割を混同しないことが重要です。
用語が増えたら、この本の第 8 章の役割表に戻ってください。

つまずき 4：Ligero の soundness 証明が見えない

Ligero は小さなランダム検査の積み重ねで健全性を作るので、
最初は “なぜこれで十分なのか” が見えにくいです。
必ず小例で membership test を体感してから原論文へ戻るとよいです。

12.8 この章のまとめ

- Ligero と STARK を学ぶには、まず骨格を掴み、その後に原論文へ入るのが効率的である。
- Ligero は **行列・RS・ランダム線形結合**、STARK は **トレース・AIR・FRI** を軸に読むとよい。
- 小さな手計算課題を通じて、記号の背後にある構造を体感することが重要である。
- 最新の実運用情報は、論文だけでなく公式ドキュメントも必ず参照する。

12.9 参考リンク集（検索しやすい題名付き）

以下は、検索用にもそのままアクセスにも使える代表リンクです。
URL は将来変わりうるため、題名でも検索できるよう併記しておきます。

- Ligero 原論文
  題名: `Ligero: Lightweight Sublinear Arguments Without a Trusted Setup`
  URL: https://acmccs.github.io/papers/p2087-amesA.pdf

- STARK 原論文
  題名: `Scalable, transparent, and post-quantum secure computational integrity`
  URL: https://starkware.co/wp-content/uploads/2022/05/STARK-paper.pdf

- FRI 原論文
  題名: `Fast Reed-Solomon Interactive Oracle Proofs of Proximity`
  URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

- Starknet 公式ドキュメント
  題名: `Introduction to Starknet's protocol`
  URL: https://docs.starknet.io/learn/protocol/intro

- StarkEx 公式ドキュメント
  題名: `Introduction :: StarkEx Documentation`
  URL: https://docs.starkware.co/starkex/index.html

必要なら、これらを章ごとの輪読資料として切り分けると学びやすくなります。

付録 用語集

本書で使った重要語を、短く引けるようにまとめます。
厳密定義ではなく、**学習の足場**としての説明を優先します。

AIR（Algebraic Intermediate Representation）

計算を、有限体上の**状態遷移多項式系**として表したもの。
STARK で「何が正しい計算か」を定義する仕様書にあたる。

ALI（Algebraic Linking IOP）

多数の AIR 制約を、ランダム係数でまとめて少数の代数的検査へ圧縮する考え方。
現代の解説では composition polynomial という言い方に吸収されることも多い。

argument

健全性が計算量仮定に依存する証明系。
ハッシュ関数の衝突耐性などの仮定のもとで安全と言う。
実用的なゼロ知識系の多くは proof ではなく argument である。

codeword（符号語）

ある符号に属する長い列。
Reed–Solomon 符号では、低次数多項式の評価列が符号語になる。

completeness（完全性）

主張が真で、証明者が正しい witness を持っているとき、検証者が受理する性質。

commitment（コミットメント）

値をいったん固定しつつ、その中身は後で必要に応じて開示できる仕組み。
Merkle 木は大量データへのコミットメントとして広く使われる。

composition polynomial

多数の制約をランダム係数で一つないし少数の多項式へまとめたもの。
正しい計算なら低次数になるよう設計される。
STARK における制約圧縮の中心概念。

execution trace（実行トレース）

計算を時系列に並べた表。
行が時刻、列がレジスタや状態変数。
STARK ではこの列が多項式として扱われる。

Fiat–Shamir 変換

公開コイン対話型プロトコルを、ハッシュ関数でチャレンジを生成することで非対話化する手法。
多くの NIZK / 非対話型 argument に使われる。

FRI（Fast Reed–Solomon Interactive Oracle Proof of Proximity）

長い評価表が低次数多項式の評価列に近いかを検査する IOPP。
STARK の中核部品の一つ。
再帰的な “折りたたみ” で問題サイズを半減させていく。

Hamming distance（ハミング距離）

二つの同長列で、値が異なる座標の数。
「コードに近い／遠い」を測る基本量。

interleaved Reed–Solomon code

複数本の RS 符号語を行として縦に積んだもの。
Ligero の主要な代数構造。
行のランダム線形結合を取りやすい。

interactive proof / interactive argument

証明者と検証者が複数回メッセージをやり取りする証明系。
Ligero も STARK も、まずは対話型として理解すると構造が追いやすい。

IOP（Interactive Oracle Proof）

大きなオラクルへのランダムアクセスと対話を組み合わせた証明モデル。
PCP より柔軟で、STARK や Ligero の理解に重要。

IOPP（Interactive Oracle Proof of Proximity）

ある長い列が、特定のコードや集合に “近い” ことを検査する IOP。
FRI は RS 符号に対する代表的 IOPP。

LDE（Low-Degree Extension）

小さな領域上で定義された列を、多項式として補間し、より大きな領域で評価し直したもの。
STARK で冗長性を増やし、低次数検査やランダム照会をしやすくする。

Merkle tree（Merkle 木）

長いデータ列に対して root hash を与える二分木型コミットメント。
任意位置の値を、短い認証パスで開示できる。

oracle

巨大な表や文字列にランダムアクセスできるブラックボックス。
実装では、Merkle root にコミットされた長い配列として実現されることが多い。

PCP（Probabilistically Checkable Proof）

証明全体を読まず、ランダムに少数箇所を見るだけで高確率に正誤判定できる証明概念。
Ligero や STARK の背景理論。

proof

健全性が計算量仮定に依らない、より強い証明概念。
実用系では strict な意味で proof でなく argument であることが多い。

public-coin

検証者の送るメッセージが、基本的にランダム文字列だけであるタイプの対話型プロトコル。
Fiat–Shamir で非対話化しやすい。

random oracle model

ハッシュ関数を理想的なランダム関数として扱う理論モデル。
Fiat–Shamir の安全性議論などで使われる。

Reed–Solomon code

低次数多項式を多数点で評価して得られる符号。
Ligero と STARK の両方に現れる基本構造。

scalability（スケーラビリティ）

証明対象が大きくなっても、証明サイズ・検証時間・証明時間の増え方が望ましいこと。
STARK の “S” が象徴する概念。

soundness（健全性）

偽の主張を、不正な証明者が高確率で受理させることができない性質。
通常は小さな誤差を伴う確率的保証として与えられる。

statement

公開される主張。
たとえば「このハッシュ値の前像を知っている」「この状態更新は正しい」など。

transparent

trusted setup や秘密 trapdoor を必要としない性質。
STARK の代表的特徴だが、Ligero も同じ方向性にある。

witness

statement を成り立たせる秘密情報。
ゼロ知識証明では、これを公開せずに条件成立だけを示す。

zero-knowledge（零知識）

検証者が、主張が真であること以外の有益な情報を得ない性質。
厳密には、シミュレータの存在で定式化される。